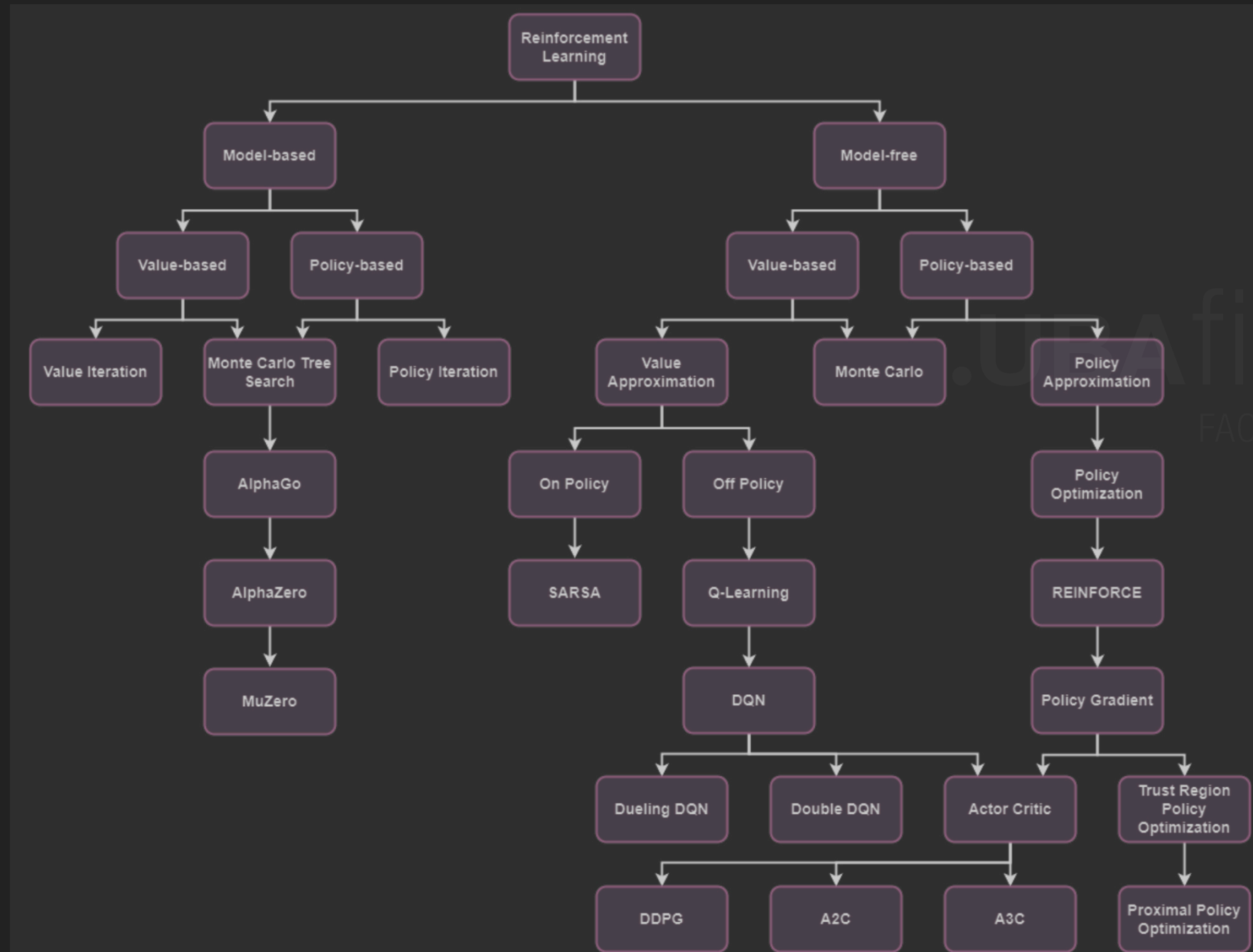


9

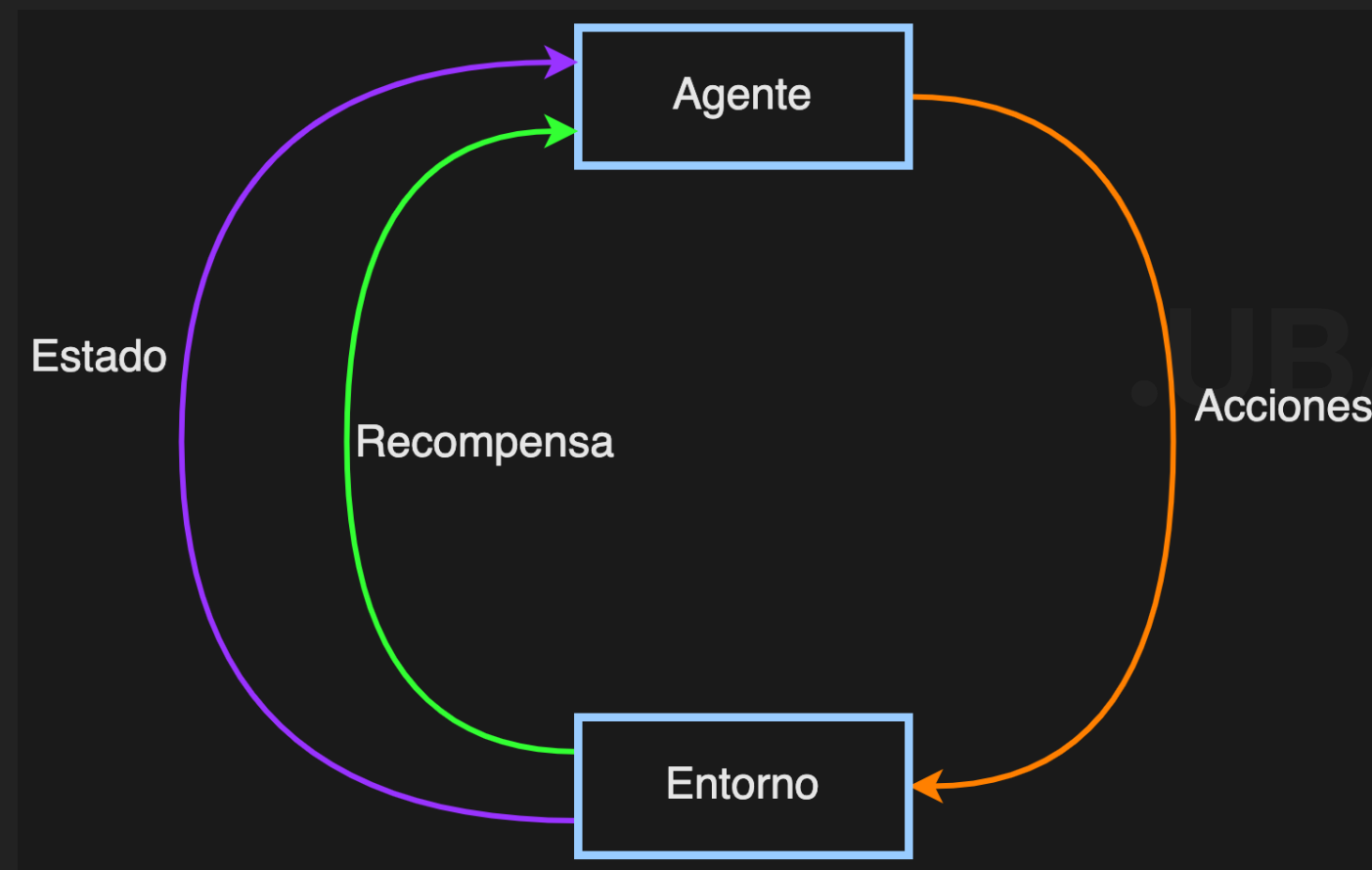
MÉTODOS DE GRADIENTE DE POLÍTICA

QUE VIMOS HASTA AHORA



CÓMO FUNCIONAN LOS MÉTODOS PG?

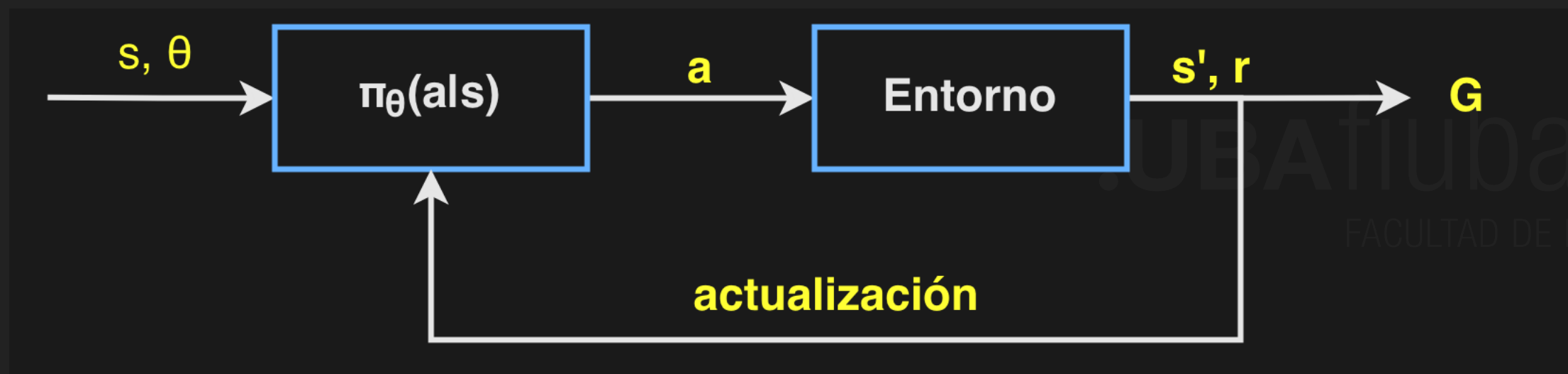
- Consideremos la estructura básica de un esquema RL:



- El entorno puede pensarse como una caja negra que procesa acciones, produce un nuevo estado y una recompensa.

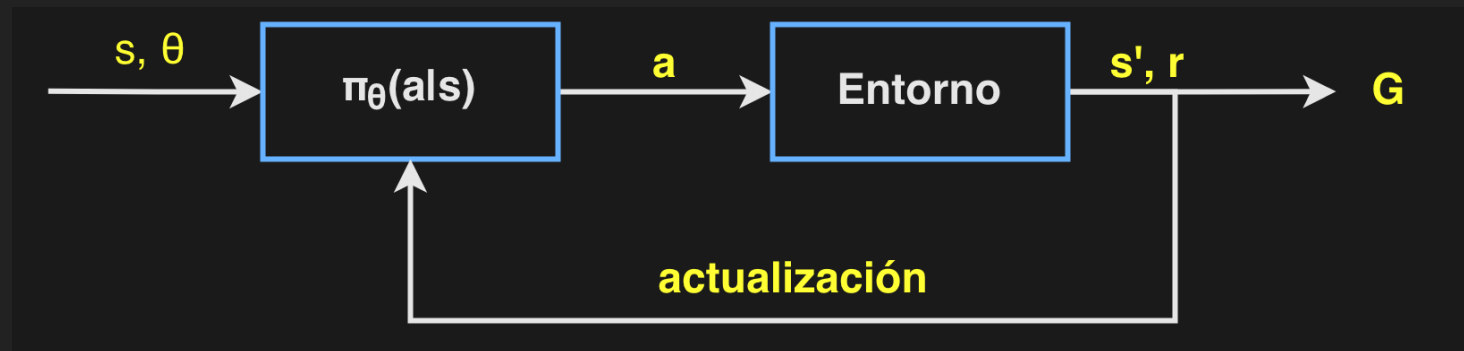
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ RL puede analizarse como si fuese un sistema de control:



- ▶ Donde: G es la recompensa acumulada

CÓMO FUNCIONAN LOS MÉTODOS PG?



- ▶ Si esto fuera deep learning clásico, tendríamos:

$$\partial G / \partial \theta$$

- ▶ aplicando regla de la cadena:

$$\partial G / \partial \theta = (\partial G / \partial s') \cdot (\partial s' / \partial a) \cdot (\partial a / \partial \theta)$$

- ▶ El problema es este término:

$$\partial s' / \partial a$$

CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ El término:

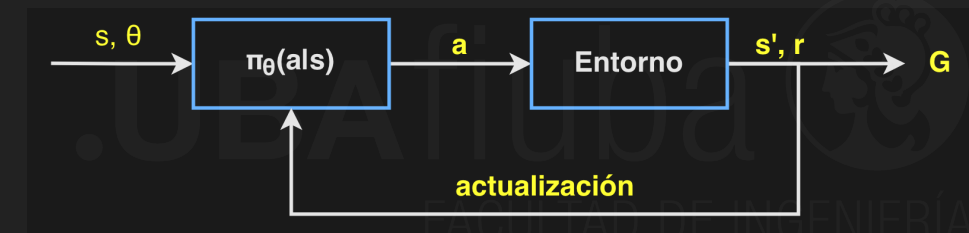
$$\partial s' / \partial a$$

- ▶ Proviene del entorno de cual:

- ✓ no se sabe su ecuación
- ✓ puede ser discreto
- ✓ puede ser no diferenciable
- ✓ o es una "caja negra"

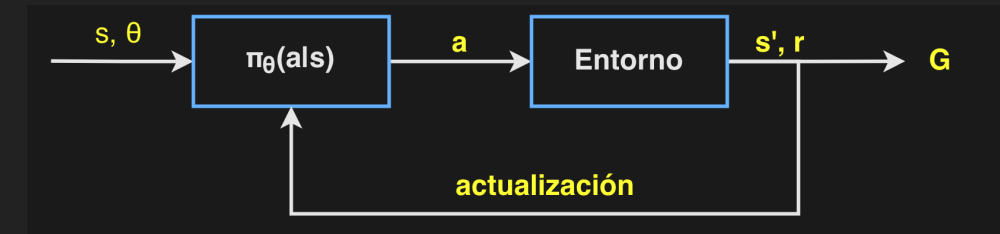
- ▶ entonces ese gradiente **no existe o no se tiene**

- ▶ **No se puede hacer backpropagation desde G hasta θ .**



CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Como se soluciona?
- ▶ En lugar de derivar



$$G(\theta)$$

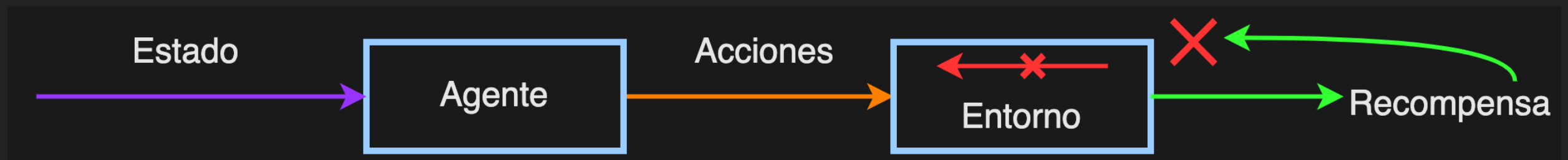
- ▶ Se deriva la probabilidad de las acciones:

$$\nabla \log \pi_{\theta}(a|s)$$

- ▶ y se usa G solo como si fuese un peso (o ponderador)

CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Si se analiza el esquema como un sistema abierto y auto-retroalimentado, el problema es que **el entorno**:
 - ✓ Realiza procesos para los cuales no tenemos una expresión o función.
 - ✓ No es diferenciable. No se puede calcular su derivada.
- ▶ Como no es posible calcular su derivada tampoco es posible **retropropagar** valores a través de él.



CÓMO FUNCIONAN LOS MÉTODOS PG?

- Esto es un problema porque el agente debe aprender (se puede considerar que el agente internamente tiene una red neuronal con pesos que deben ser actualizados).



CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Para actualizar los pesos se necesita retropropagar a través de la red neuronal. Motivo por el cual se necesita el **gradiente** a la salida de la red.



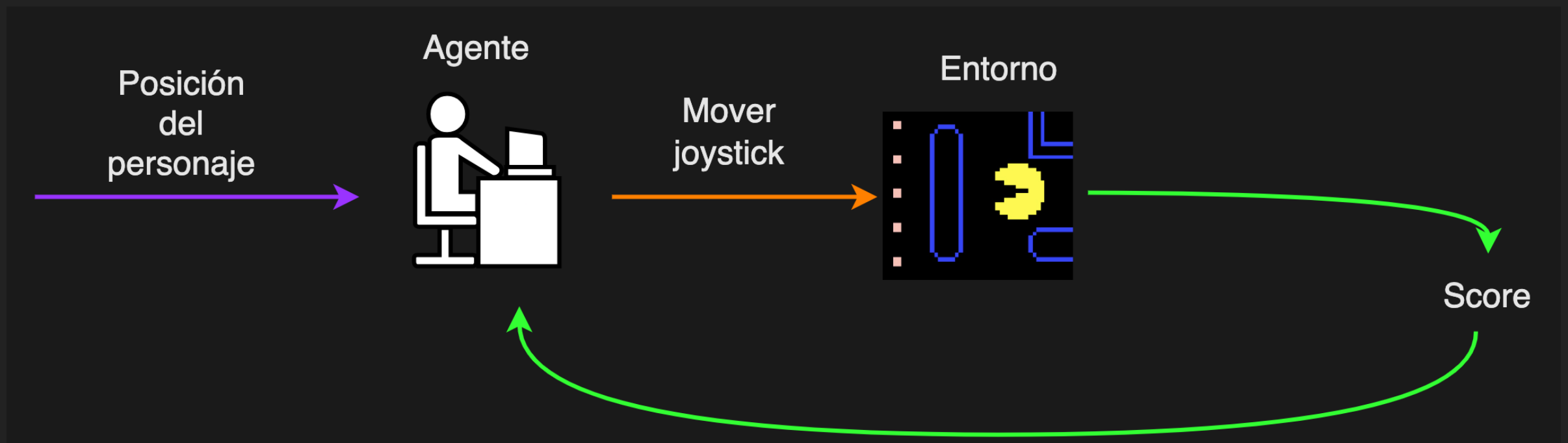
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ En lugar de usar la salida del entorno para intentar retropropagar a través de él y luego tratar de retropropagar a través del agente para actualizar los pesos de la red, se debería encontrar **otras formas de actualizar los pesos**.



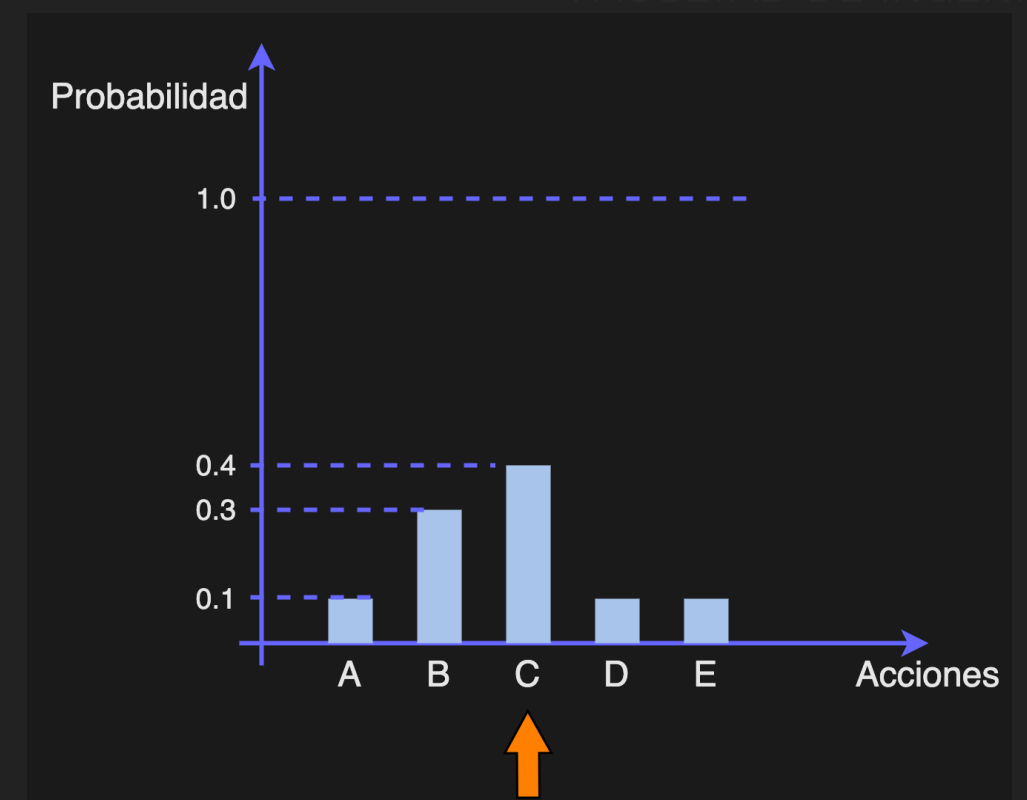
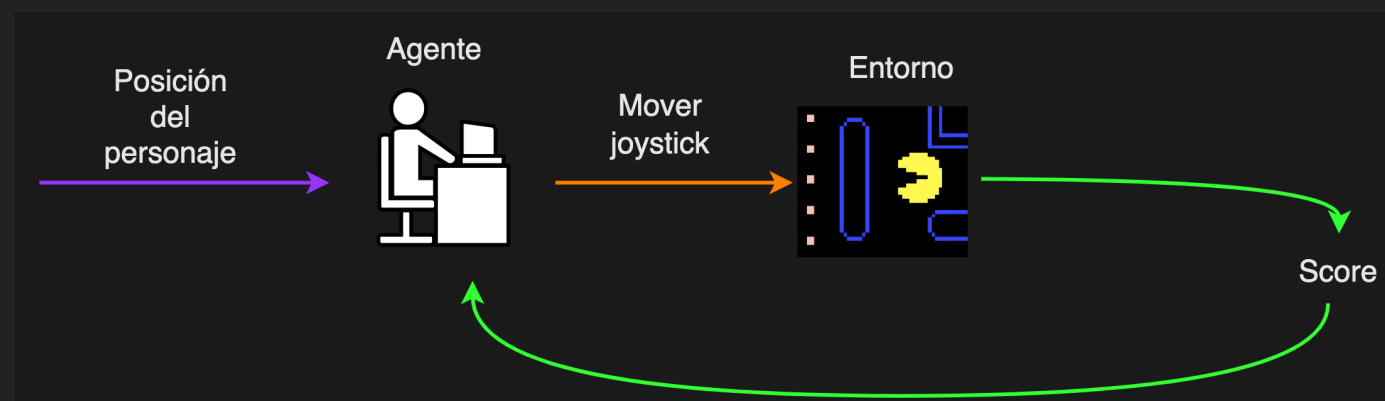
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Imaginemos una persona intentando aprender a ganar en un videojuego.
- ▶ La persona realizará una acción en el juego que le otorga un puntaje positivo y por eso la persona tenderá a recordar esta acción y hará esa acción **más a menudo**.
- ▶ La persona **usa el puntaje recibido para reforzar** aquellas acciones que el entorno considera buenas.



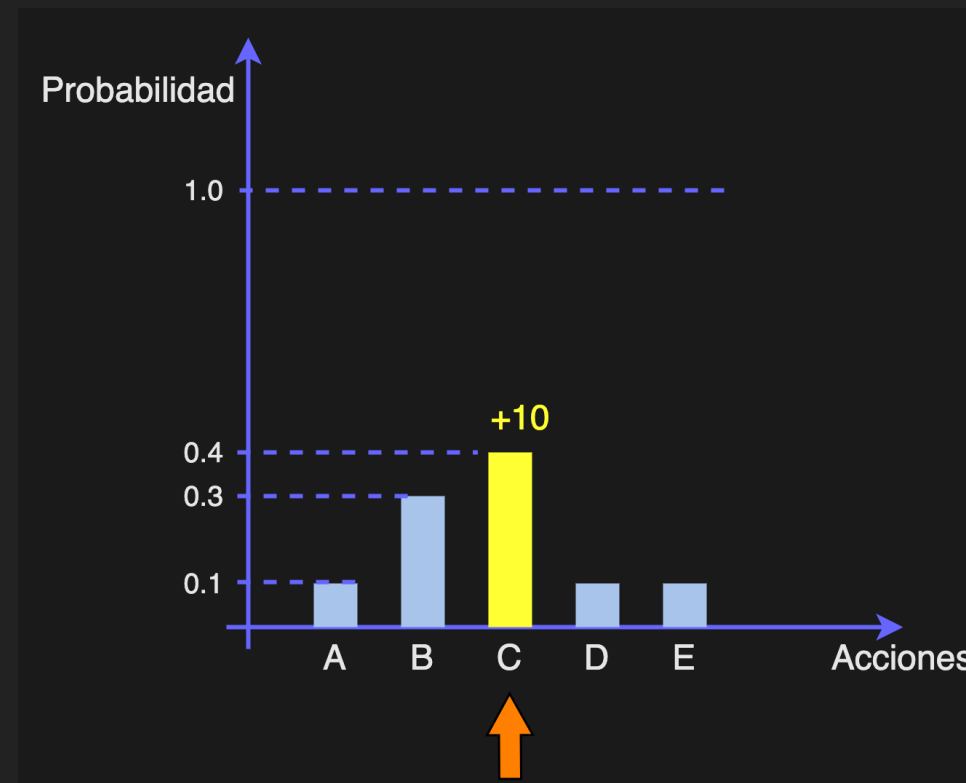
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Es decir, si la persona selecciona una **acción** que redunde en un buen resultado, entonces debería tomar esa acción más a menudo en el futuro.
- ▶ Se podría pensar que existe una **distribución de acciones** posibles que puede tomar la persona mientras juega.



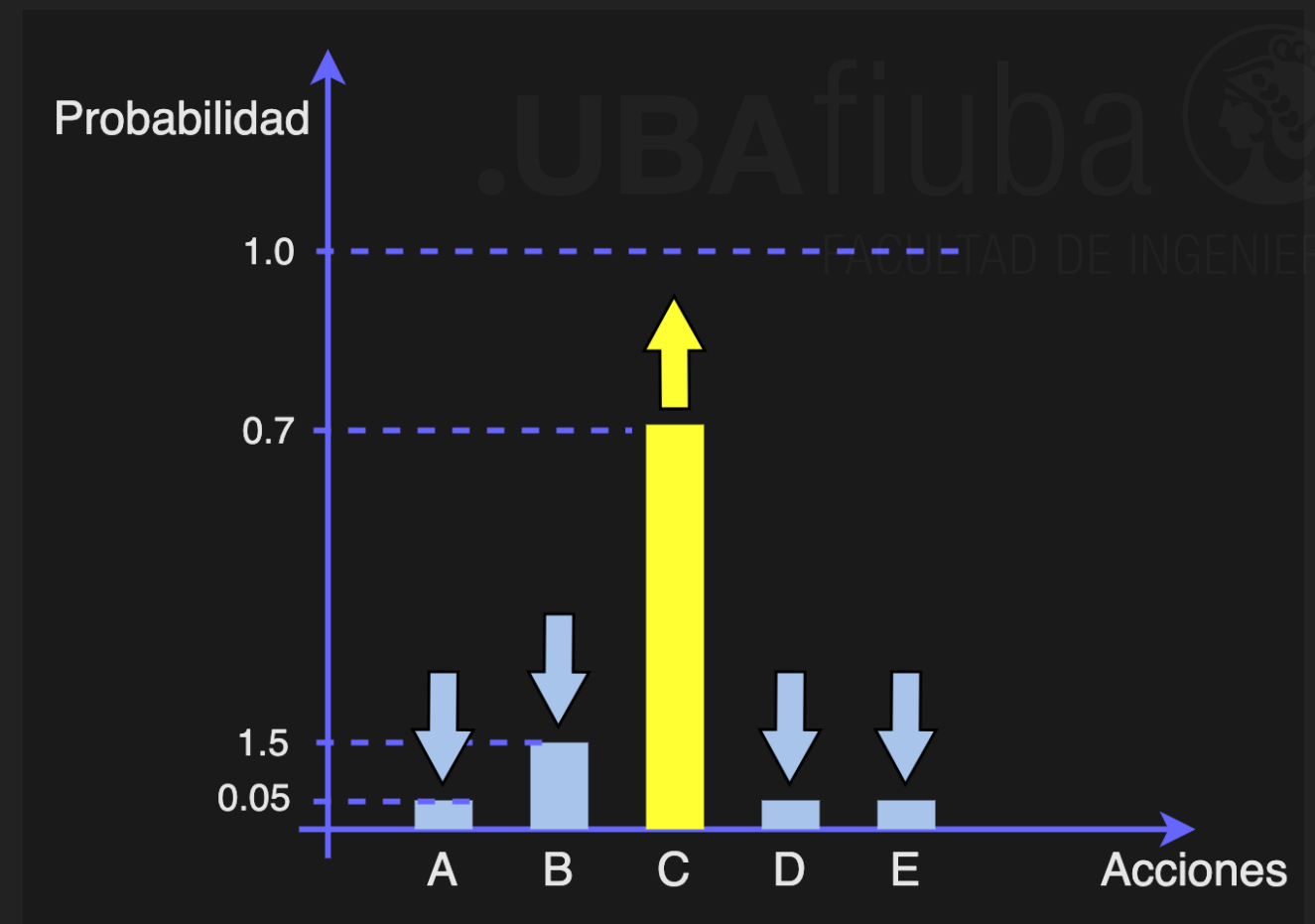
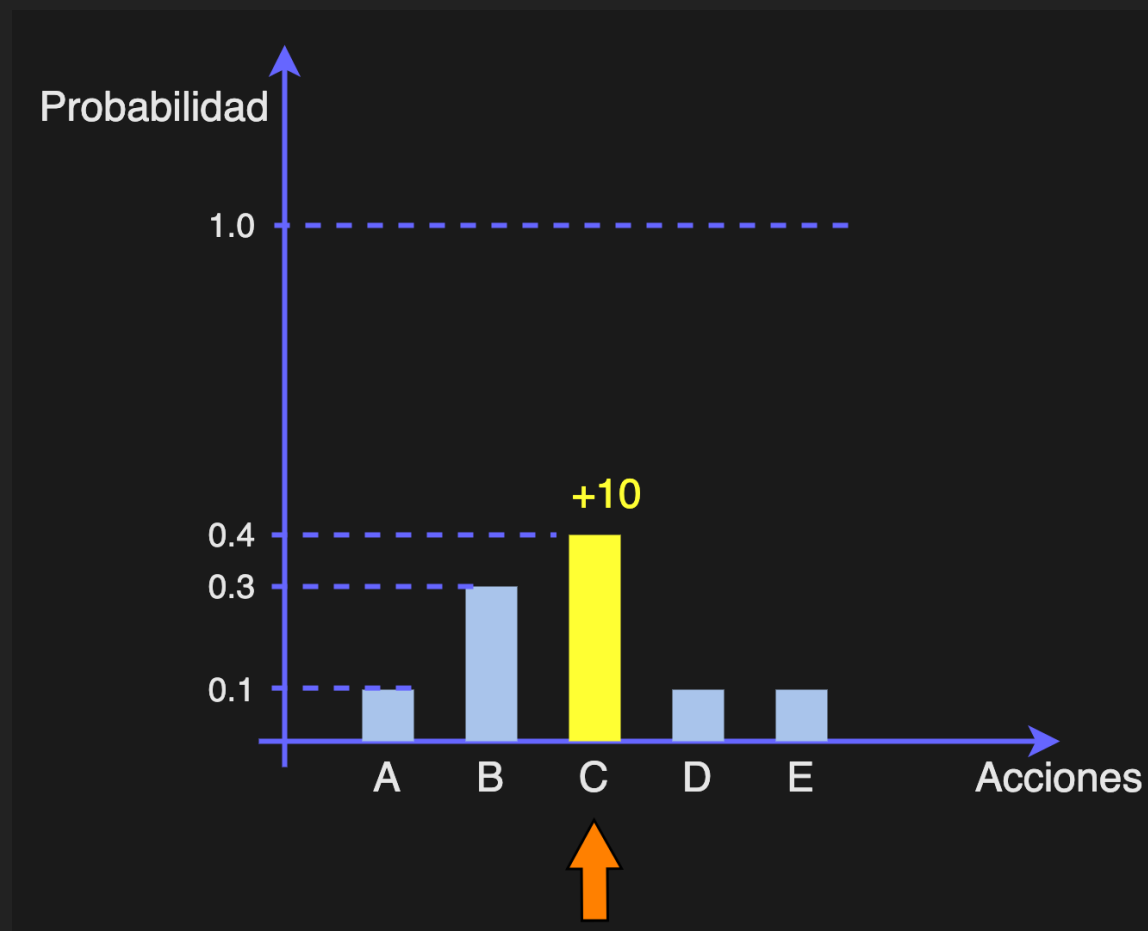
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ Si en esta **distribución de acciones** posibles una de estas acciones resulta llevar a un buen resultado (devuelve una recompensa grande) entonces habría que **"aplastar"** lateralmente la distribución alrededor de esa acción, de modo que esa acción ocurra más a menudo.



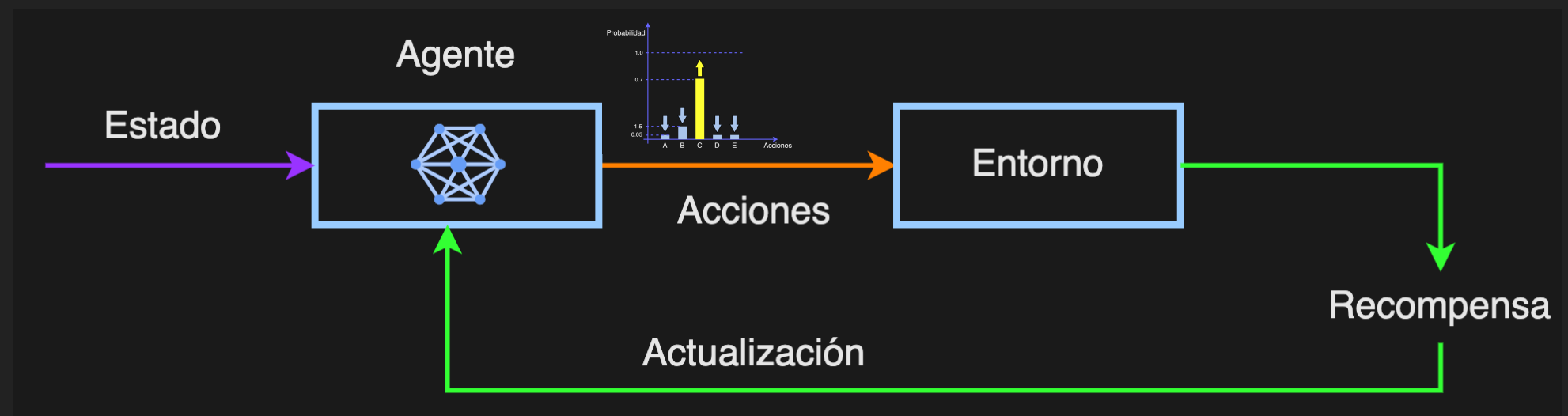
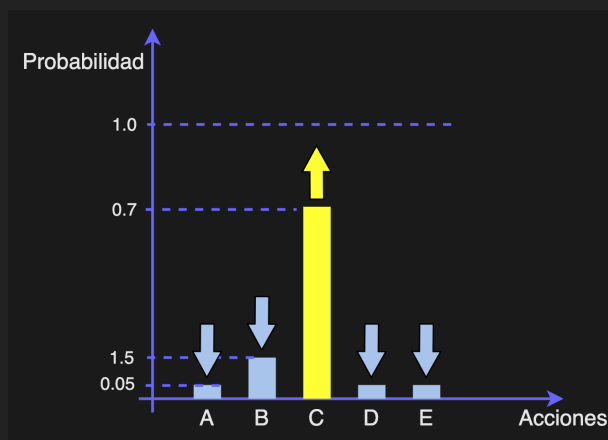
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ La acción C ahora tiene mayor probabilidad dado que la “**empujamos**” hacia arriba y bajamos las demás.



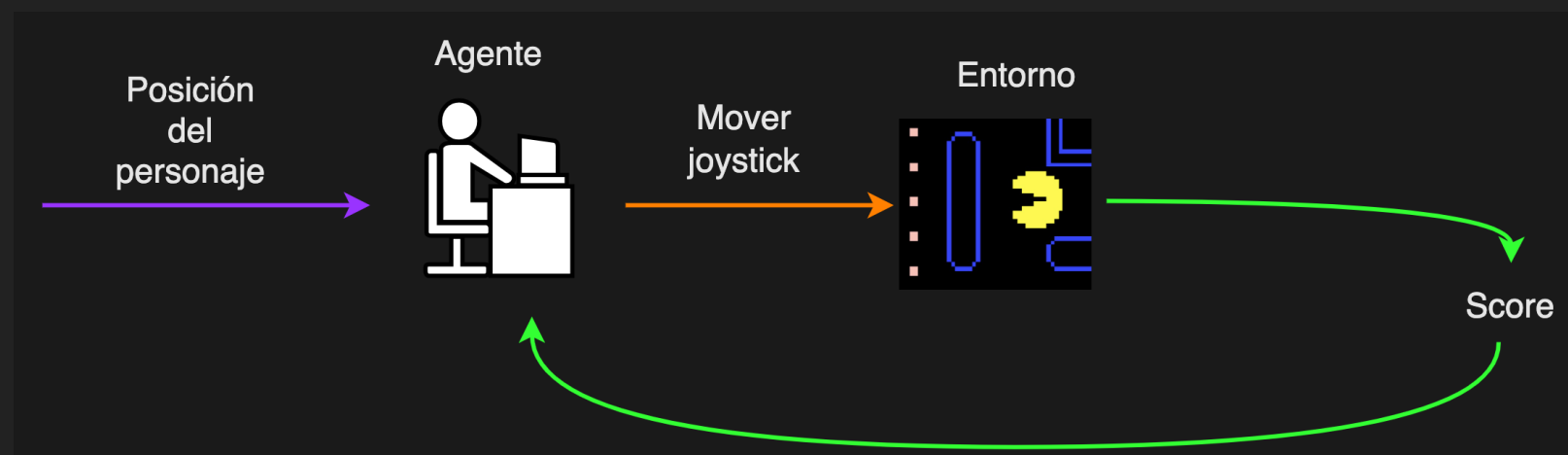
CÓMO FUNCIONAN LOS MÉTODOS PG?

- ▶ De forma más general, se puede usar un resultado bueno o malo como señal para el aprendizaje.
- ▶ Esto se puede usar siempre que exista alguna medida de 'bondad' a la salida de la red neuronal.
- ▶ Cuando no se puede retropropagar o diferenciar a través del proceso que entrega la salida, en lugar de usar el gradiente descendente como se hace normalmente en aprendizaje supervisado, se puede **ajustar el gradiente ascendente del comportamiento (la política) en la misma dirección de la recompensa**.
- ▶ Esta es la idea de los **Métodos de Gradiente de Política**. Es decir, si se tiene algo que no se puede diferenciar, se usa la **recompensa** como señal para ayudar a **cambiar los pesos**.



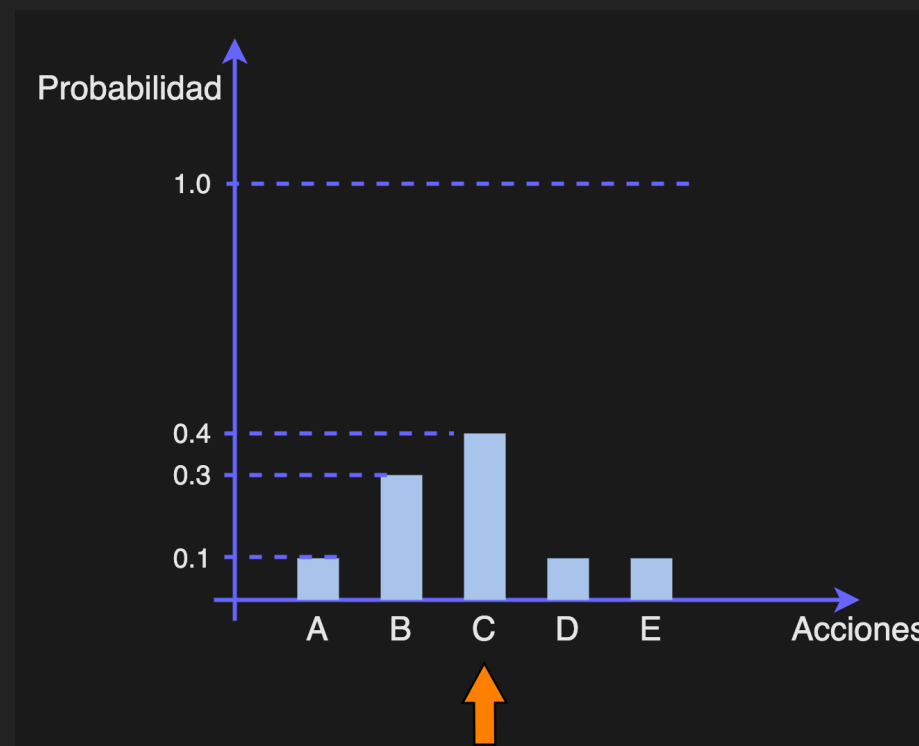
CONCEPTO GENERAL DE PG

- ▶ Los métodos de aprendizaje como SARSA, Q-Learning y DQN utilizan funciones de valor $V(s)$ o $Q(s, a)$.
- ▶ En los métodos de **gradiente de política (Policy Gradient)**, se aproxima directamente la política óptima sin necesidad de usar una función de valor.
- ▶ Ya no se considera el **valor de las acciones** para que el agente aprenda, sino que el agente actuará según una política que se irá optimizando gradualmente.



CONCEPTO GENERAL DE PG

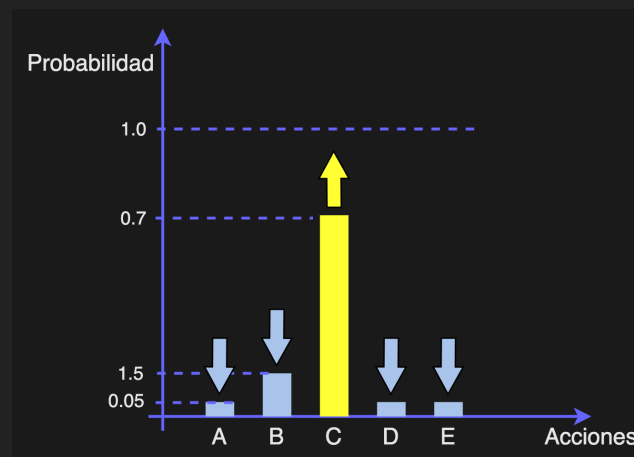
- ▶ En los métodos de **valor de acción** (como Q-Learning), el agente aprende qué tan buena es cada **acción a** en un **estado s** particular. Estima ese valor. Luego, basándose en esos valores aprendidos, **deduce una política** (una forma de comportarse).
- ▶ Los **métodos de gradiente de la política** aprenden una política **parametrizada por θ** (los pesos de una red neuronal). Esto es, se comportan como una función que **mapea estados a probabilidades de acciones**. Aprenden la estrategia en sí, no solo el valor de las partes de la estrategia.



CONCEPTO GENERAL DE PG

- ▶ Estos métodos usan parámetros, tales como θ (theta), para definir la política. Ejemplo: la notación $\pi(a|s, \theta)$, es la probabilidad de tomar la acción 'a' en el estado 's' y dados esos parámetros de política θ .
- ▶ Se llaman métodos de gradiente de política porque aprenden los parámetros de la política siguiendo el gradiente (la dirección de mayor aumento de alguna medida de rendimiento). Esta medida de rendimiento se denomina $J(\theta)$.
- ▶ Es un ascenso por gradiente, intentando maximizar el rendimiento. La actualización de los pesos implica tomar los parámetros antiguos θ y sumarles un tamaño de paso α multiplicado por el gradiente de $J(\theta)$.

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$

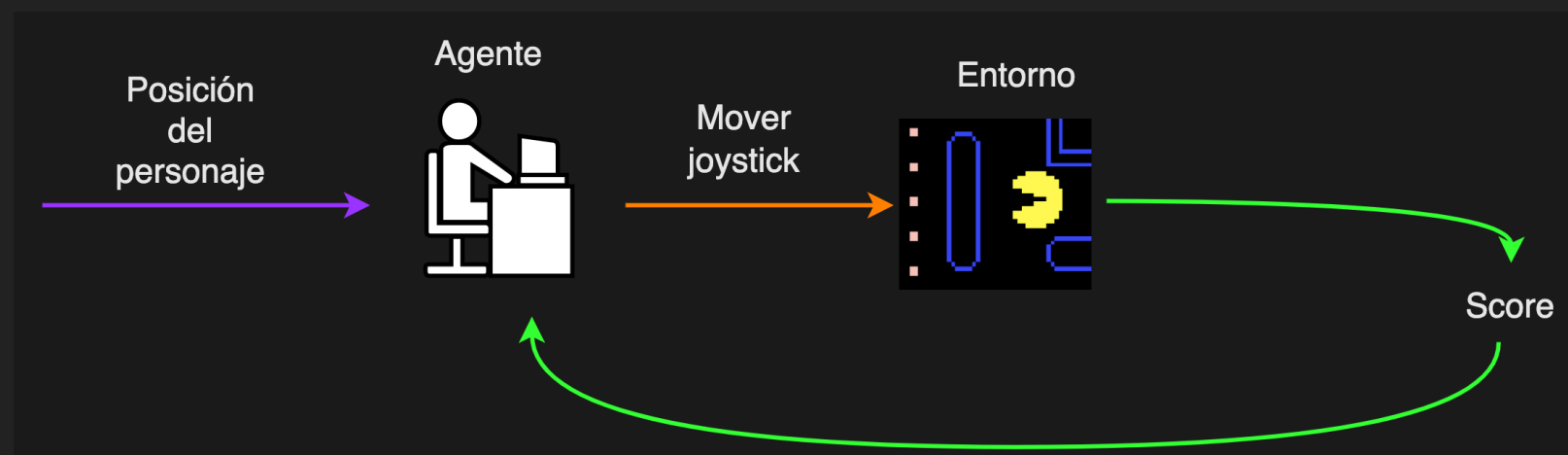
 $V(s)$ $Q(s, a)$

$$A(s, a) = Q(s, a) - V(s)$$

 $J(\theta)$

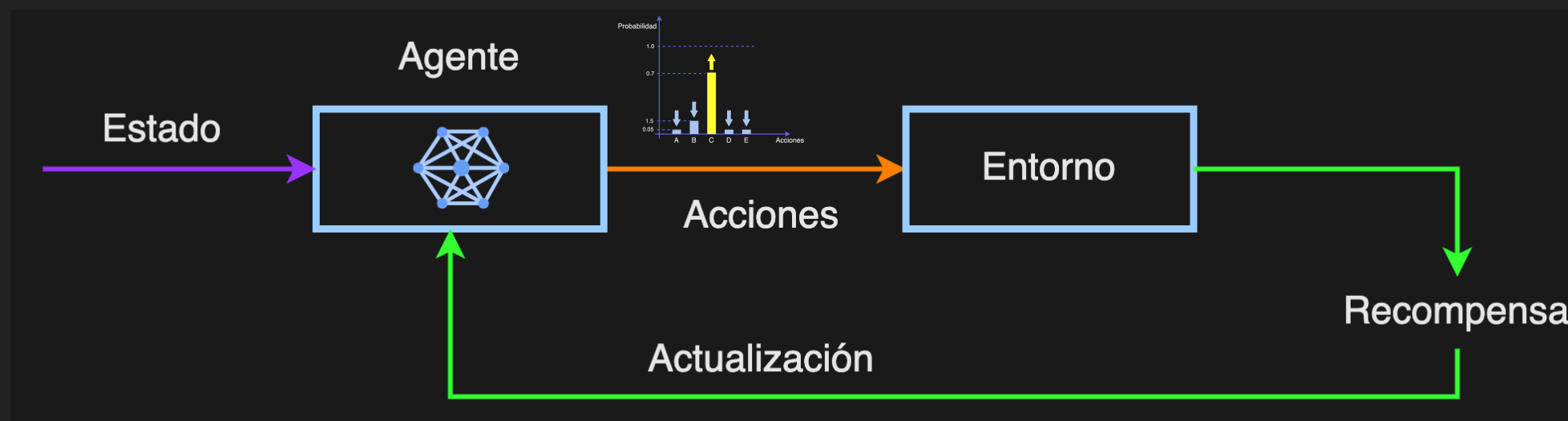
GRADIENTES DE POLÍTICA

- ▶ Otra forma de que un agente aprenda, en lugar de usar una función de valor, es expresar la política directamente sin necesidad de una función de valor.
- ▶ En general los métodos PG (Policy Gradient), la política es independiente de la función de valor (es así en métodos REINFORCE, no en Actor-Critic).
- ▶ El agente aprende la política π_{θ} , la cual indica qué acción tomar en un estado s , sin requerir una función de valor.



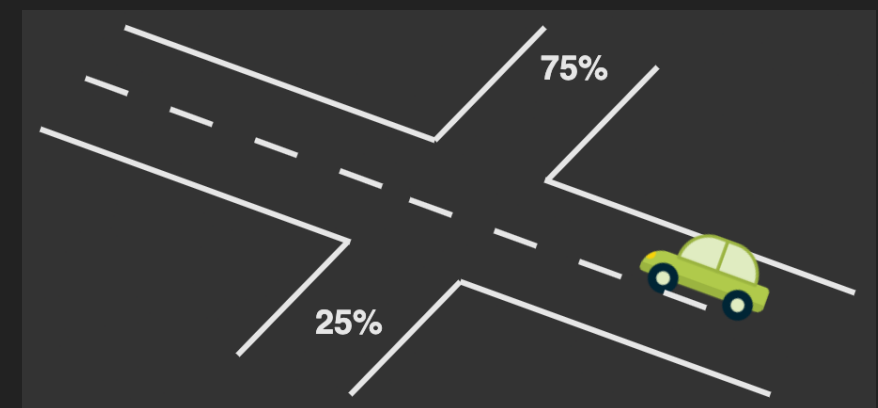
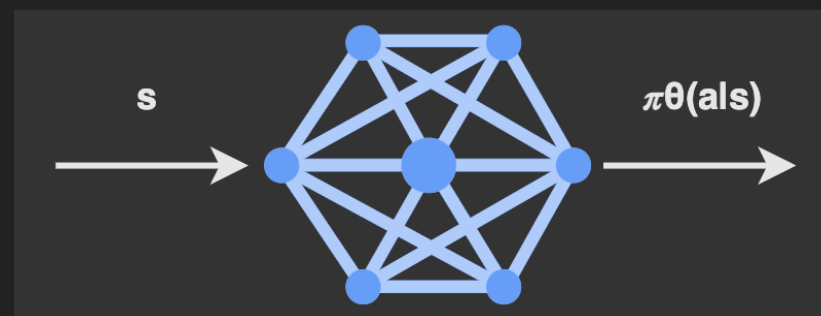
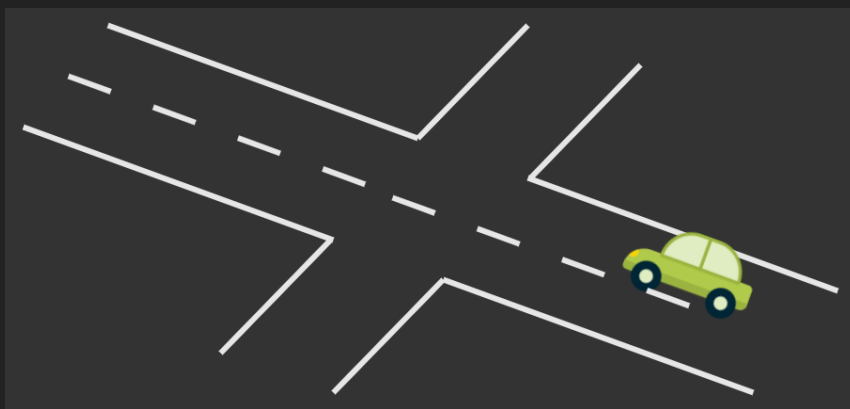
GRADIENTES DE POLÍTICA

- ▶ Al igual que en **Q-learning**, el objetivo sigue siendo obtener la máxima recompensa, con la diferencia de que **ahora el agente sigue una política parametrizada que depende de θ** .
- ▶ La probabilidad de tomar cada acción para un determinado estado la genera una **red neuronal**.
- ▶ En cada interacción del agente en el entorno la red neuronal:
 - ✓ Tiene a su entrada el estado actual,
 - ✓ Ajusta sus pesos θ para que las mejores acciones tengan mas probabilidad de ser tomadas y,
 - ✓ Entrega en su salida una distribución de probabilidad de las acciones.



EJEMPLO: CONDUCCIÓN DE UN COCHE

- ▶ El vehículo puede moverse a la derecha o a la izquierda, pero doblar a la izquierda recibe menos recompensa.
- ▶ Para cada estado, la política π_{θ} asignará más probabilidad a la mejor acción.
- ▶ En cada estado s se produce una salida de la red hasta que finalice el episodio.
- ▶ Al final del episodio se actualizan los θ de la red (a diferencia de los métodos de función de valor en los que las actualizaciones se producían luego de cada acción)

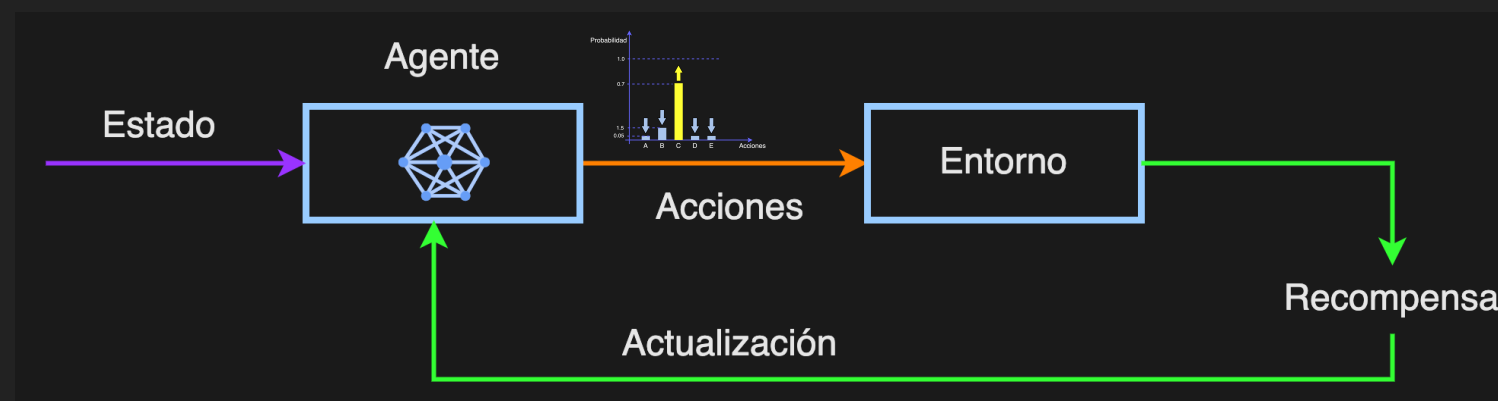


EJEMPLO: CONDUCCIÓN DE UN COCHE

- ▶ La red deja que el **agente** tome **acciones** libremente pero en cada episodio se ajusta y otorga al agente probabilidades de acciones cada vez mejores.
- ▶ Durante el episodio, la red genera acciones libremente, pero al final del episodio se actualizan los parámetros θ . Esto es típico en **REINFORCE**.
- ▶ En cada nuevo episodio, la red ajusta sus parámetros para dar mejores probabilidades de acción.
- ▶ La política se parametriza así:

$$\pi(a|s, \theta) = P(A_t = a \mid S_t = s, \theta_t = \theta)$$

se lee, la probabilidad de que el agente tome la **acción a** en el **tiempo t** dado un **estado s** en el mismo **tiempo t** y dado el vector de parámetros θ .



ESPACIO DE LAS ACCIONES

► El espacio de acciones bajo la distribución de probabilidad parametrizada por θ puede ser:

✓ **Discreto**: la distribución de probabilidad es **categorica***, y la salida suele pasar por una función **softmax****.

*Una **distribución categorica** es una distribución de probabilidad que se usa cuando se tienen varias clases o categorías posibles, y solo una de ellas puede ocurrir a la vez.

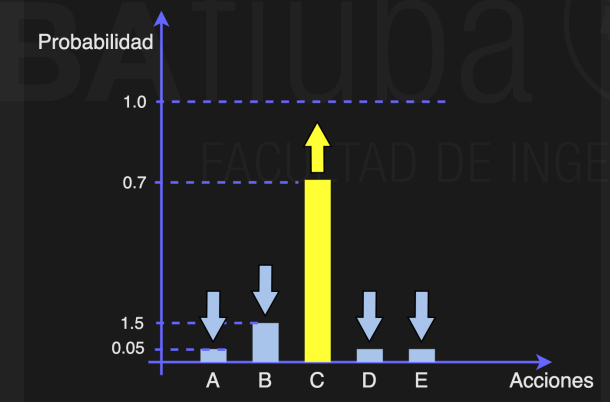
La función **softmax es una función matemática que convierte un vector de números reales (por ejemplo, puntuaciones o logits) en una distribución de probabilidad.

Ejemplo: Dado un vector $Z=[z_1, z_2, \dots, z_n]$ la función softmax devuelve:

$$\text{Softmax}(z_i) = e^{z_i} / \sum(e^{z_j})$$

La función softmax:

- ✓ Realza el valor más grande y suprime los pequeños.
- ✓ Da más probabilidad al valor más alto, pero aún mantiene un "reparto" suave entre todos.
- ✓ Se usa para representar la probabilidad de que una clase sea la correcta, especialmente en problemas de clasificación multiclase (como MNIST, CIFAR-10, etc.).
- ✓ **Continuo**: la salida está definida por los parámetros de una distribución continua (por ejemplo, la media y la varianza de una distribución gaussiana).

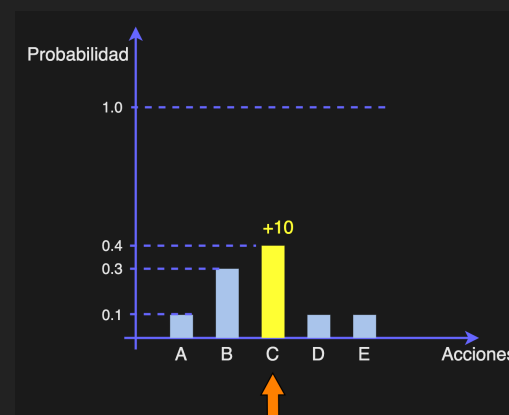


TIPOS DE POLÍTICAS

- ▶ **Determinista**: para un estado s , la política $\pi(s)$ devuelve directamente la acción a que maximiza la recompensa.
- ▶ **Estocástica**: para un estado s , la política $\pi(a|s)$ devuelve una distribución de probabilidad continua del conjunto de acciones a tomar. Es decir que:

$$\pi(a|s, \theta) = P(A_t = a \mid S_t = s, \theta_t = \theta)$$

- ▶ Esta distribución continua de probabilidad permite que una política estocástica se aplique en espacios continuos.
- ▶ El objetivo de un algoritmo PG es ajustar una red neuronal parametrizada por θ , para que aprenda una política estocástica $\pi(a|s, \theta)$, representada mediante una distribución de probabilidad que mapea estados con acciones, buscando maximizar una función objetivo $J(\theta)$.



FUNCIÓN OBJETIVO $J(\theta)$

- Caso **discreto**:

Policy Gradient

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$$

Monte Carlo

$$v_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]$$

donde $J(\theta) = v_{\pi_{\theta}}(S_0)$ es el valor esperado de las recompensas descontadas para la trayectoria completa τ (tau) desde el estado inicial S_0 y siguiendo una política π_{θ} .

En PG no interesa el valor de un estado en particular, pero sí interesa el rendimiento promedio de toda la política.

- Caso **continuo** o de **horizonte infinito** (PG en entornos continuos sin episodios y sin descuento):

$$J(\theta) = \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=0}^h \mathbb{E}[r(s_t, a_t) \mid A_{0:t} \sim \pi_{\theta}]$$

$$\lim_{t \rightarrow \infty} \mathbb{E}[r(s_t, a_t) \mid A_{0:t} \sim \pi_{\theta}]$$

- $J(\theta)$ = promedio de recompensas por paso a medida que $t \rightarrow \infty$

ACTUALIZACIÓN DE θ

- ▶ En Deep Learning se minimiza una función de pérdida. En Policy Gradient se maximiza el rendimiento $J(\theta)$ mediante ascenso por gradiente:

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$

donde $\nabla J(\theta_t) \in \mathbb{R}^{d'}$ es una estimación estocástica y α es el coeficiente de aprendizaje. Es decir que el gradiente de J es un vector de la misma dimensión que θ .

- ▶ La actualización y optimización de θ se hace en la misma dirección del gradiente mediante el ascenso del gradiente.

TEOREMA DE GRADIENTE DE POLÍTICA

- ▶ Vimos anteriormente que la función objetivo en el caso discreto era:

$$J(\theta) = v_{\pi_{\theta}}(S_0)$$

donde $v_{\pi_{\theta}}(S_0)$ es el valor de la función para π_{θ} y S_0 es el estado inicial.

- ▶ Si se maximiza $J(\theta)$ se está maximizando $v_{\pi_{\theta}}$.

- ▶ También vimos que la actualización es:

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$

entonces:

$$\nabla J(\theta_t) = \nabla v_{\pi_{\theta}}(S_0)$$

TEOREMA DE GRADIENTE DE POLÍTICA

- ▶ Como a y s dependen del entorno (desconocido), no se puede calcular directamente el $\nabla_{\theta} v_{\pi_{\theta}}(S_0)$.
- ▶ La solución a esto es aplicar el **teorema de gradientes de politica** que expresa el rendimiento J respecto a la politica (Sutton et al., 2018):

$$\nabla J(\theta) = \sum_s \mu(s) \sum_a q_{\pi}(s, a) \nabla \pi(a|s, \theta)$$

donde:

- ✓ $\mu(s)$: distribución de estados bajo la política π
 - ✓ $q_{\pi}(s, a)$: valor de acción de la política
 - ✓ $\nabla \pi(a|s, \theta)$: gradiente de la política
- ▶ Si se busca mejorar la política se debe cambiar θ en la dirección donde más aumentan las probabilidades de tomar acciones buenas (las que tienen alto $q_{\pi}(s, a)$)
 - ▶ Este teorema es model-free (no requiere conocer el modelo del entorno).

TEOREMA DE GRADIENTE DE POLÍTICA

► No conozco el gradiente de π , es decir, $\nabla \pi(a|s, \theta)$, pero lo puedo escribir como:

$$\nabla_{\theta} \pi(a|s, \theta) = \pi(a|s, \theta) \cdot \frac{\nabla_{\theta} \pi(a|s, \theta)}{\pi(a|s, \theta)} = \pi(a|s, \theta) \cdot \nabla_{\theta} \ln \pi(a|s, \theta)$$

reemplazando el gradiente de pi:

$$\nabla J(\theta) = \sum_s \mu(s) \sum_a q_{\pi}(s, a) \pi(a|s, \theta) \nabla_{\theta} \ln \pi(a|s, \theta)$$

Expresado en términos de la esperanza:

$$\nabla J(\theta) = \mathbb{E}_{s \sim \mu, a \sim \pi_{\theta}} [q_{\pi}(s, a) \nabla_{\theta} \ln \pi(a|s, \theta)]$$

Sabemos que:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

luego G_t es la suma de recompensas descontadas:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$\mathbb{E}_{\pi} [G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta)]$$

Finalmente reemplazo el gradiente de J en la actualización theta:

$$\nabla J(\theta) = \mathbb{E}_{\pi} [G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta)]$$

$$\theta_{t+1} = \theta_t + \alpha G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta)$$

REINFORCE



REINFORCE

- El algoritmo **REINFORCE** fue propuesto por (Williams, 1992).

Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning

Ronald J. Williams
College of Computer Science
Northeastern University
Boston, MA 02115

Appears in *Machine Learning*, 8, pp. 229-256, 1992.

Abstract

This article presents a general class of associative reinforcement learning algorithms for connectionist networks containing stochastic units. These algorithms, called REINFORCE algorithms, are shown to make weight adjustments in a direction that lies along the gradient of expected reinforcement in both immediate-reinforcement tasks and certain limited forms of delayed-reinforcement tasks, and they do this without explicitly computing gradient estimates or even storing information from which such estimates could be computed. Specific examples of such algorithms are presented, some of which bear a close relationship to certain existing algorithms while others are novel but potentially interesting in their own right. Also given are results that show how such algorithms can be naturally integrated with backpropagation. We close with a brief discussion of a number of additional issues surrounding the use of such algorithms, including what is known about their limiting behaviors as well as further considerations that might be used to help develop similar but potentially more powerful reinforcement learning algorithms.

REINFORCE

- ▶ Es un algoritmo Monte Carlo (no usa bootstrap).
- ▶ Espera al final del episodio para observar retornos completos y actualizar θ .
- ▶ Para aplicar el teorema PG se necesita:
 - ✓ una política parametrizada $\pi(a|s, \theta)$
 - ✓ muestreo de acciones A_t en cada instante t

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

REINFORCE CON BASELINE

- ▶ El problema de REINFORCE es que G_t (el retorno) es muy variable:

- ✓ Puede ser muy grande o muy chico dependiendo de la trayectoria.
- ✓ Esto genera ruido en el gradiente → dificulta aprender.

- ▶ Para reducir esa variabilidad, se introduce un término baseline $b(S_t) = V(s)$ y se actualiza usando:

$$(G_t - b(S_t)) \nabla_{\theta} \ln \pi(A_t | S_t, \theta)$$

- ✓ G_t mide "qué tan bien salió la trayectoria completa".
- ✓ $b(S_t)$ actúa como un "promedio esperado" en ese estado S_t .
- ✓ Entonces $G_t - b(S_t)$ mide si lo que realmente pasó fue mejor o peor de lo que normalmente pasa.

- ▶ **Resultado:**

- ✓ Si la acción fue mejor de lo esperado, la política se refuerza.
- ✓ Si fue peor de lo esperado, la política se debilita.
- ✓ La varianza se reduce, pero no se introduce sesgo (no cambia el valor esperado del gradiente).

REINFORCE CON BASELINE

► REINFORCE con Baseline utiliza dos redes neuronales:

- ✓ Una red para la política π_{θ} .
- ✓ Una red para el baseline V_w .

► Dos optimizadores separados: uno para θ , otro para w (aprende el valor esperado de un estado).

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, w)$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^w > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$ (e.g., to 0)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\delta \leftarrow G - \hat{v}(S_t, w)$$

$$w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S_t, w)$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta)$$

ACTOR-CRITIC



ACTOR-CRITIC

- ▶ Analogía: Un **chef (crítico)** y un **aprendiz (actor)**.
- ▶ El **aprendiz** al inicio utilizará los utensilios incorrectos, quemará lo que intenta cocinar, utilizará proporciones de ingredientes inexactas, etc.
- ▶ El **chef** observará, corregirá o lo felicitará (otorga el valor de **Q** o recompensa según la acción).
- ▶ Ambos modelos aprenden (no solo el aprendiz).
- ▶ El **crítico** aprende a juzgar una **acción** comparando continuamente sus propias predicciones de valor con los resultados observados.

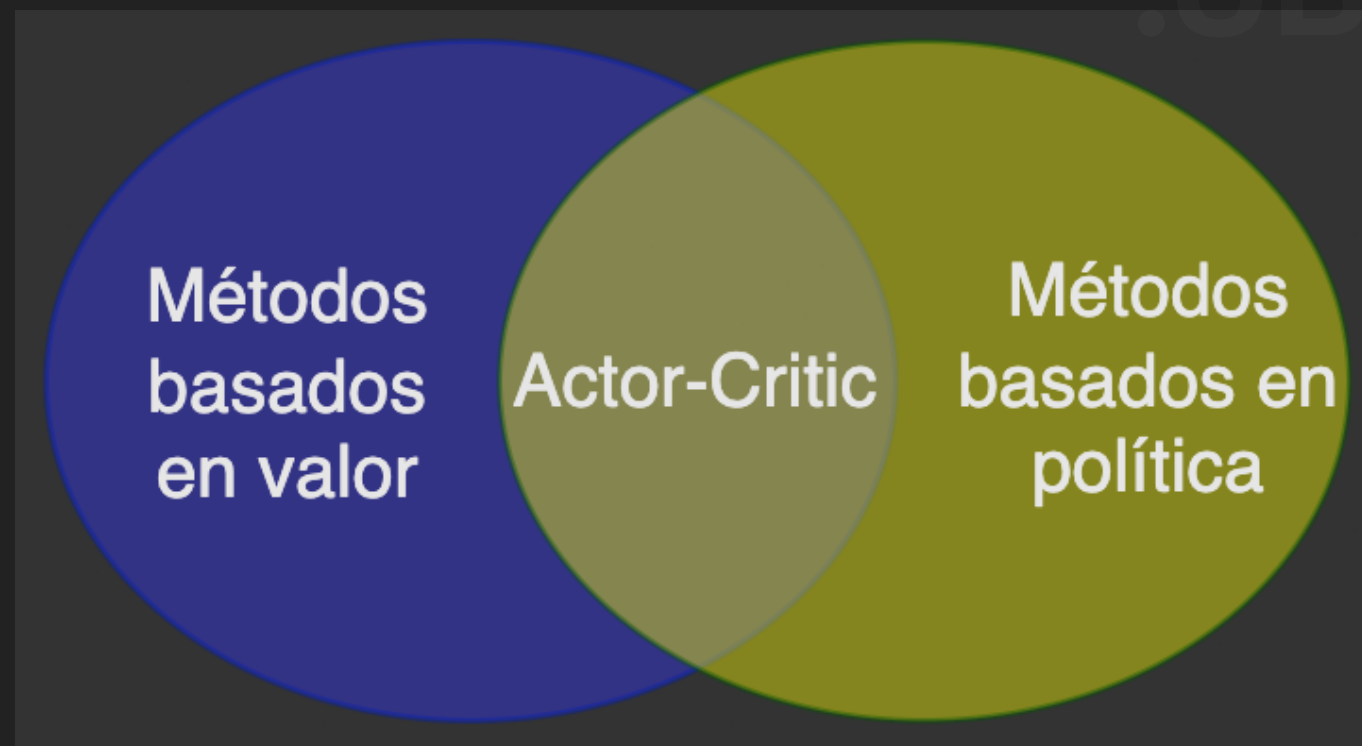


ACTOR-CRITIC

- ▶ Hasta ahora vimos 2 tipos de métodos de RL libres de modelos:
 - ✓ Basados en la función de valor (Q-Learning, DQN)
 - ✓ Basados en la política (REINFORCE)
- ▶ Ventajas de cada método:
 - ✓ Basados en valor -> mas estables y eficientes.
 - ✓ Basados en política -> mejores en espacios estocásticos y continuos.
- ▶ ¿conviene combinarlos para extraer lo mejor ambos métodos? SI

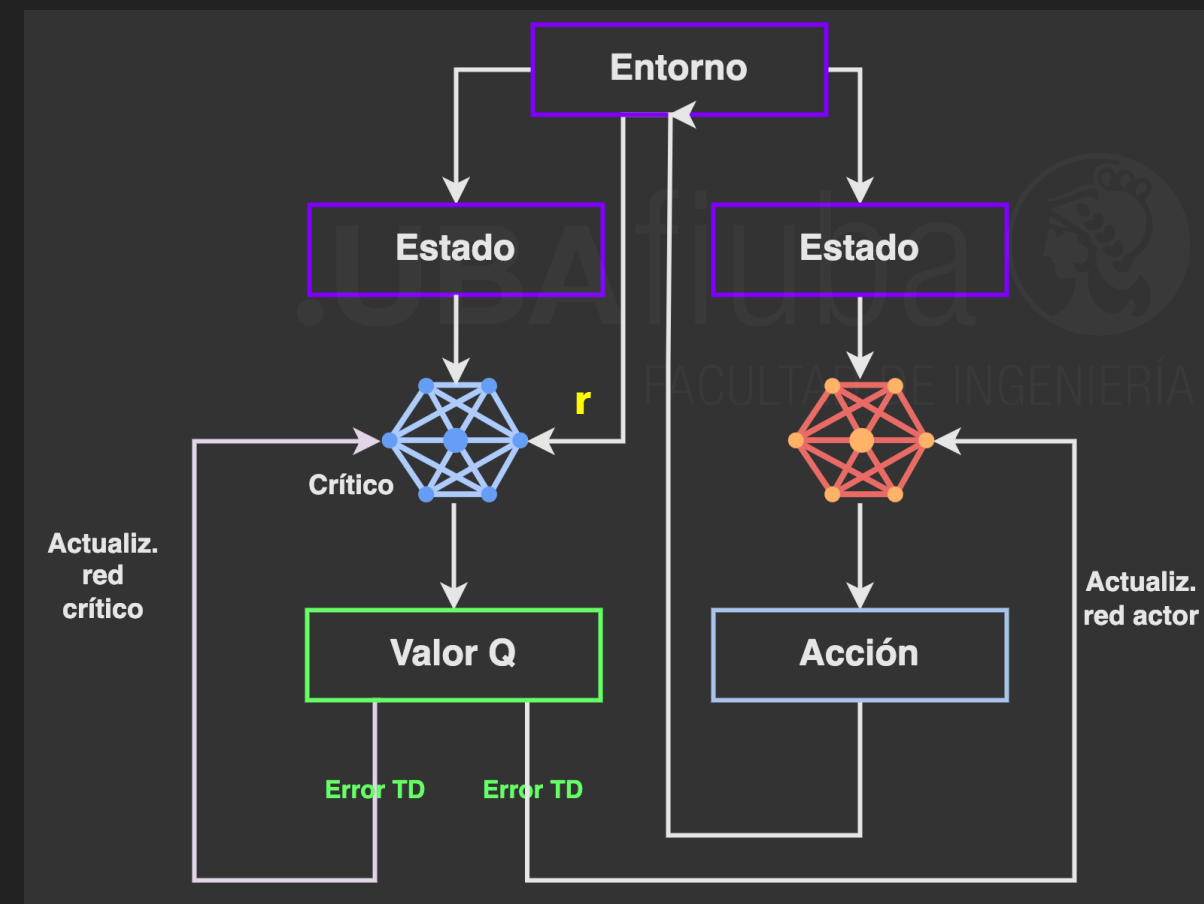
ACTOR-CRITIC

- ▶ El algoritmo **Actor-Critic** combina ambos métodos, esto es, emplea la función de valor y también la política para determinar la mejor acción en cada estado.
- ▶ A-C aprende explícitamente tanto la política (el actor) como una función de valor (el crítico), que a menudo estima los valores de estado.



ACTOR-CRITIC

- ▶ El modelo se divide en 2:
 - ✓ Una parte que determina la acción según el estado.
 - ✓ Y la otra parte determina el valor de Q de la acción.
- ▶ Para implementarlo se utilizan 2 redes neuronales:
 - ✓ **Actor**: es el que determina qué acción tomar en cada estado. Utiliza una función de aproximación que esta basada en la política.
 - ✓ **Critico**: es quien evalúa lo buena o lo mala que resultó la acción seleccionada para el estado actual. Utiliza una función de aproximación que esta basada en valor (Q).



ACTOR-CRITIC

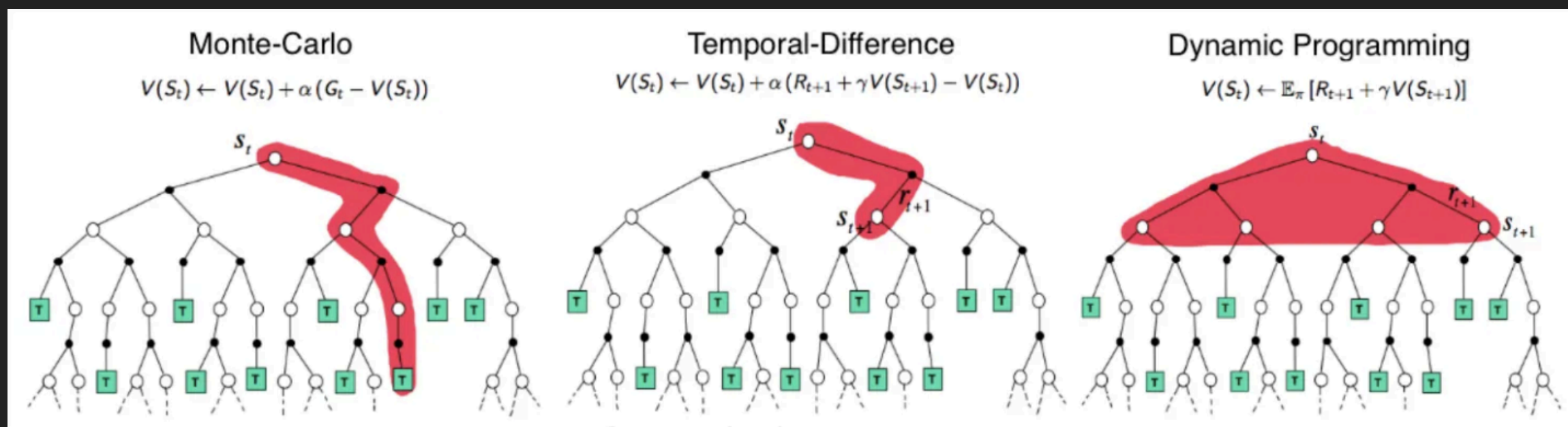
- ▶ El algoritmo **Actor-Critic** combina ambos métodos, esto es, emplea la función de valor y también la política para determinar la mejor acción en cada estado.
- ▶ A-C aprende explícitamente tanto la política (el actor) como una función de valor (el crítico), que a menudo estima los valores de estado.

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

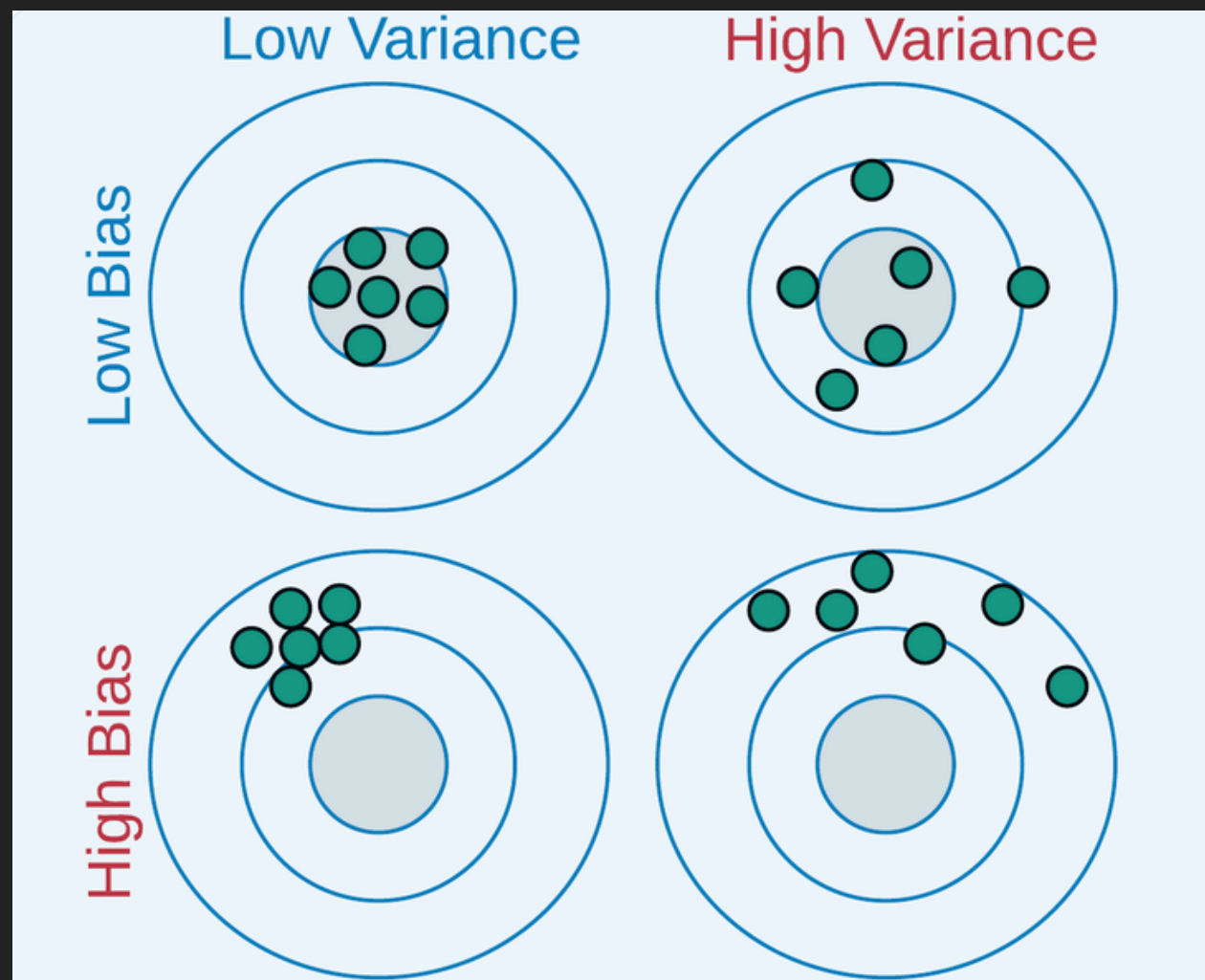
ESTIMACIÓN DE $V(S)$

- ▶ **Monte Carlo:** espera hasta el final de un episodio para obtener el retorno real G_t . Luego, usa este retorno completo y real para actualizar el valor de todos los estados visitados en ese episodio (S_t incluido).
- ▶ **TD:** actualiza la estimación del valor de S_t después de cada paso, sin esperar al final del episodio. Lo hace usando la recompensa real inmediata R_{t+1} y la propia estimación actual del valor del siguiente estado $V(S_{t+1})$ (bootstrapping).
- ▶ **Programación Dinámica:** requiere un modelo completo del entorno (es decir, conocer las probabilidades de transición $P(s'|s, a)$ y las recompensas esperadas $R(s, a, s')$). Actualiza el valor de un estado basándose en los valores calculados (usando el modelo) de todos los posibles estados sucesores.



VARIANZA Y BIAS

- ▶ La **varianza** es medir cuánto las estimaciones se parecen entre sí.
- ▶ El **bias** es medir cuan alejada esta la estimación del valor real.



RETORNOS

► Los retornos para estimar $V(s)$ son: one-step, n-step, Monte Carlo ("T-step").



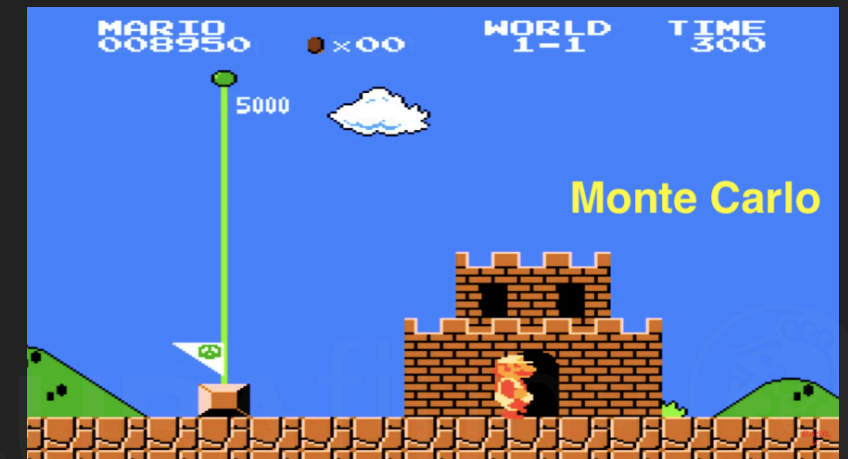
► Diferencia Temporal

► Actor-Critic

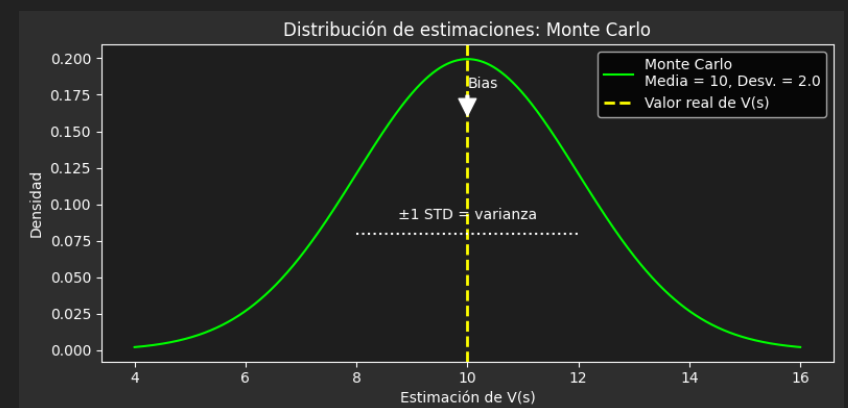
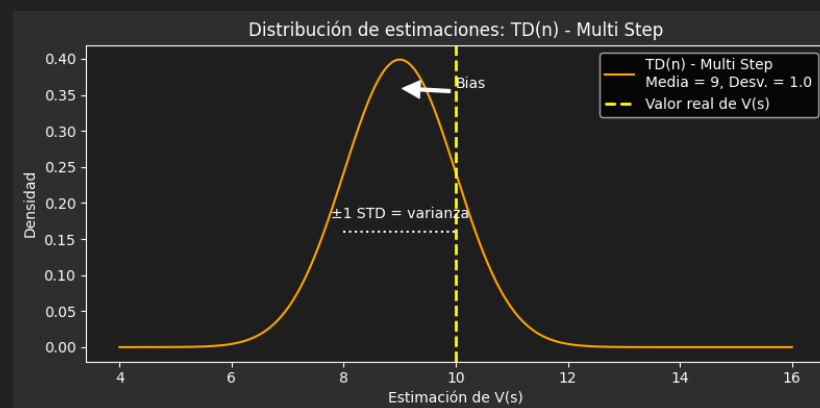
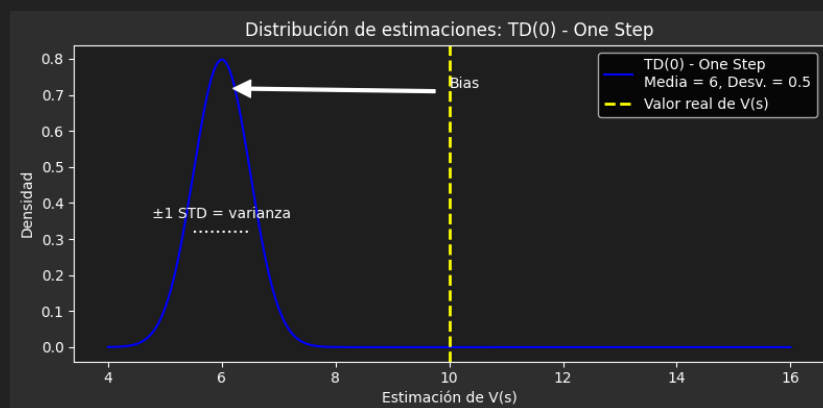


► Actor-Critic

► Advantage Actor-Critic (A2C)



► Monte Carlo



$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} +$$

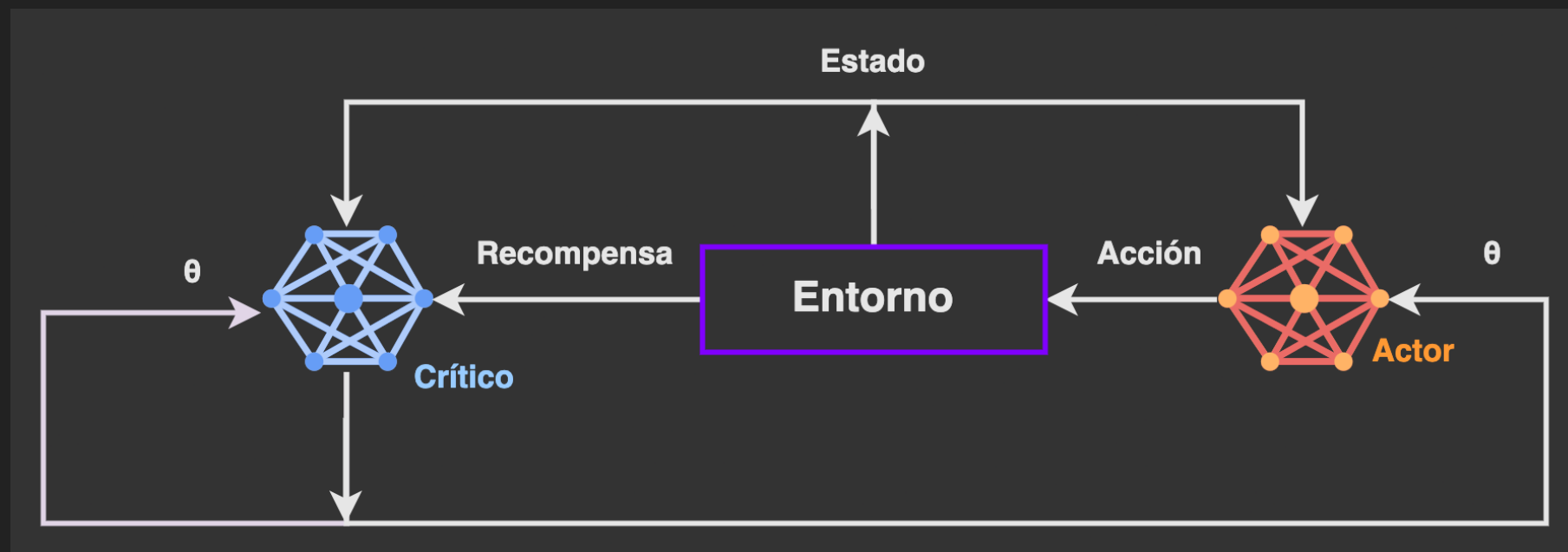
$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

ACTOR-CRITIC

- ▶ Objetivo: encontrar una política (una forma de actuar) óptima (π^*) para el agente.
- ▶ Componentes:

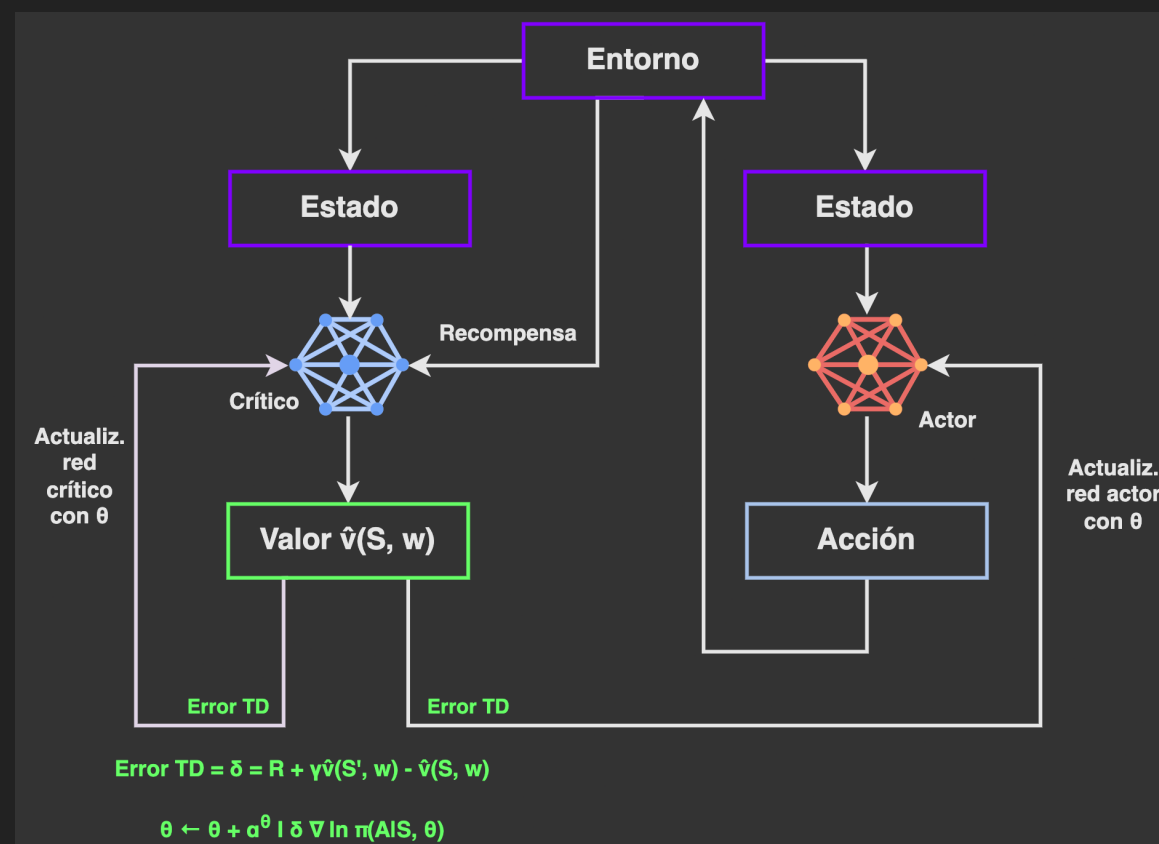
✓ Actor

✓ Crítico



ACTOR-CRITIC

- ✓ **Actor:** Red neuronal que decide qué acción tomar en un estado dado. Está parametrizada por su pesos θ . Su objetivo es mejorar la política $\pi(a|s, \theta)$ para que elija acciones que lleven a mayores recompensas acumuladas.
- ✓ **Crítico:** Red neuronal que evalúa qué tan buena es una acción o un estado. Estima la función de valor de estado $\hat{v}(s, w)$, que predice la recompensa acumulada esperada a partir de un estado s . Está parametrizado por w . Ayuda al Actor diciéndole si las acciones tomadas fueron mejores o peores de lo esperado.



ACTOR-CRITIC

- ▶ **One-step**: Las actualizaciones se basan en la información del siguiente paso inmediato (la recompensa R y el valor estimado del siguiente estado S').
- ▶ Esto se conoce como aprendizaje por Diferencia Temporal de orden cero, **TD(0)**. Si fuese **n-step** se lo suele denominar **TD(λ)**.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► Inputs:

- ✓ $\pi(a|s, \theta)$, política del Actor. Debe ser diferenciable respecto a θ para poder calcular gradientes. Define la probabilidad de tomar la acción a en el estado s .
- ✓ $\hat{v}(s, \mathbf{w})$: es la función de valor de estado estimada por el Crítico. Debe ser diferenciable respecto a $\mathbf{w}$. Estima el valor (retorno esperado) del estado s .

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► Parameters:

- ✓ $\alpha_\theta > 0, \alpha_w > 0$: Tasas de aprendizaje (o tamaños de paso) para el Actor (θ) y el Crítico (w), respectivamente. Controlan cuanto se actualizan los parámetros en cada paso.

One-step Actor-Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, w)$
 Parameters: step sizes $\alpha^\theta > 0, \alpha^w > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', w) - \hat{v}(S, w)$ (if S' is terminal, then $\hat{v}(S', w) \doteq 0$)
 $w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S, w)$
 $\theta \leftarrow \theta + \alpha^\theta I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► Initialize (Inicialización):

- ✓ Se inicializan los parámetros θ del Actor y los pesos w del Crítico (a menudo con ceros o valores aleatorios pequeños). d' y d son las dimensiones de estos vectores de parámetros.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, w)$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^w > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', w) - \hat{v}(S, w)$ (if S' is terminal, then $\hat{v}(S', w) \doteq 0$)
 $w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S, w)$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► Initialize S (first state of episode):

✓ Se obtiene el estado inicial S para el nuevo episodio.

► $I \leftarrow 1$:

✓ Se inicializa una variable I . Esta variable acumula el factor de descuento (γ) a lo largo del episodio (I representará γ^t en el paso t). Empieza en 1 porque $\gamma^0 = 1$.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Initialize S (first state of episode)

$I \leftarrow 1$

Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

$S \leftarrow S'$

ACTOR-CRITIC

- ▶ Loop while S is not terminal (for each time step) = Bucle interno (por cada paso de tiempo dentro del episodio):
- ✓ Este bucle se ejecuta mientras el agente no haya llegado a un estado final.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► $A \sim \pi(\cdot|S, \theta)$:

- ✓ El Actor elige una acción A basándose en la política actual π para el estado actual S . La acción se muestra (samplea) de la distribución de probabilidad definida por la política.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► Take action A , observe S' , R :

✓ El agente ejecuta la acción A en el entorno. Como resultado, observa el siguiente estado S' y recibe una recompensa inmediata R .

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

```

Input: a differentiable policy parameterization  $\pi(a|s, \theta)$ 
Input: a differentiable state-value function parameterization  $\hat{v}(s, \mathbf{w})$ 
Parameters: step sizes  $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$ 
Initialize policy parameter  $\theta \in \mathbb{R}^{d'}$  and state-value weights  $\mathbf{w} \in \mathbb{R}^d$  (e.g., to  $\mathbf{0}$ )
Loop forever (for each episode):
  Initialize  $S$  (first state of episode)
   $I \leftarrow 1$ 
  Loop while  $S$  is not terminal (for each time step):
     $A \sim \pi(\cdot|S, \theta)$ 
    Take action  $A$ , observe  $S', R$ 
     $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$       (if  $S'$  is terminal, then  $\hat{v}(S', \mathbf{w}) \doteq 0$ )
     $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$ 
     $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$ 
     $I \leftarrow \gamma I$ 
     $S \leftarrow S'$ 
  
```

ACTOR-CRITIC

- ▶ $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$: Cálculo del Error TD (Temporal Difference) (sirve para calcular los pesos $\mathbf{w}$ del Crítico).
- ▶ $R + \gamma \hat{v}(S', \mathbf{w})$: Es el objetivo TD (TD target). Es una estimación mejorada del valor del estado S , basada en la recompensa real R obtenida y el valor estimado del siguiente estado S' (descontado por γ).
- ▶ $\hat{v}(S, \mathbf{w})$: Es la estimación actual del valor del estado S según el Crítico.
- ▶ δ : El Error TD. Mide la diferencia entre el objetivo TD (lo que debería ser el valor de S según la experiencia de un paso) y la estimación actual del Crítico.
- ▶ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$): Si el siguiente estado es terminal, su valor es cero por definición (no hay más recompensas futuras).

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

- ▶ $w \leftarrow w + \alpha w \delta \nabla \hat{v}(S, w)$: Actualización de los pesos w del Crítico
- ▶ $\nabla \hat{v}(S, w)$: Es el gradiente de la función de valor estimada con respecto a sus pesos w , evaluado en el estado actual S .
- ▶ La actualización mueve los pesos w en la dirección que reduce el error TD (δ). Si δ es positivo (el resultado fue mejor de lo esperado), $\hat{v}(S, w)$ aumentará. Si δ es negativo (peor de lo esperado), $\hat{v}(S, w)$ disminuirá. El objetivo es que $\hat{v}(S, w)$ se acerque al objetivo $R + \gamma \hat{v}(S', w)$.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, w)$
 Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^w > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', w) - \hat{v}(S, w)$ (if S' is terminal, then $\hat{v}(S', w) \doteq 0$)
 $w \leftarrow w + \alpha^w \delta \nabla \hat{v}(S, w)$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

- ▶ $\theta \leftarrow \theta + \alpha \theta \mid \delta \nabla \ln \pi(A|S, \theta)$: Actualización del Actor
- ▶ $\nabla \ln \pi(A|S, \theta)$: Es el gradiente del logaritmo de la probabilidad de la política para la acción específica A que se tomó en el estado S . Este gradiente apunta en la dirección que aumenta la probabilidad de tomar esa acción A en el estado S .
- ▶ δ : El error TD calculado por el Crítico actúa como una señal de ventaja. Si $\delta > 0$ (la acción llevó a un resultado mejor de lo esperado), la actualización aumentará la probabilidad de tomar la acción A en el estado S . Si $\delta < 0$ (peor de lo esperado), disminuirá esa probabilidad.
- ▶ I : El factor de descuento acumulado (γ^t). Pondera la actualización, dando potencialmente menos importancia a las decisiones tomadas más tarde en el episodio (aunque esto depende del valor de γ). El Actor usa la evaluación (δ) del Crítico para saber en qué dirección ajustar su política.

One-step Actor-Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^\theta > 0, \alpha^\mathbf{w} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^\theta I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► $I \leftarrow \gamma I$:

- ✓ Se actualiza el factor de descuento acumulado para el siguiente paso de tiempo ($\gamma^{t+1} = \gamma * \gamma^t$).

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0, \alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

ACTOR-CRITIC

► $S \leftarrow S'$:

✓ El estado actual se actualiza al siguiente estado observado.

One-step Actor-Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

$S \leftarrow S'$

CUANDO USAR ACTOR-CRITIC?

- ▶ Cuando se maneja **espacios de estado continuos** (las RNA como aproximadoras de función toman vectores de números reales tales como posiciones, velocidades, ángulos, etc., como entrada).
- ▶ La variante de Actor-Critic A2C también se adapta a **estados discretos**.
- ▶ El Actor puede manejar **acciones continuas** o los parámetros de una distribución de probabilidad de acciones continuas.
- ▶ Cuando la **exploración es crítica** (ej: entornos con recompensas escasas).
- ▶ Cuando la **estabilidad es más importante que el rendimiento**. Es menos eficiente que sus variantes pero es más estable en algunos casos porque evita complejidades como Replay buffers y Target networks.

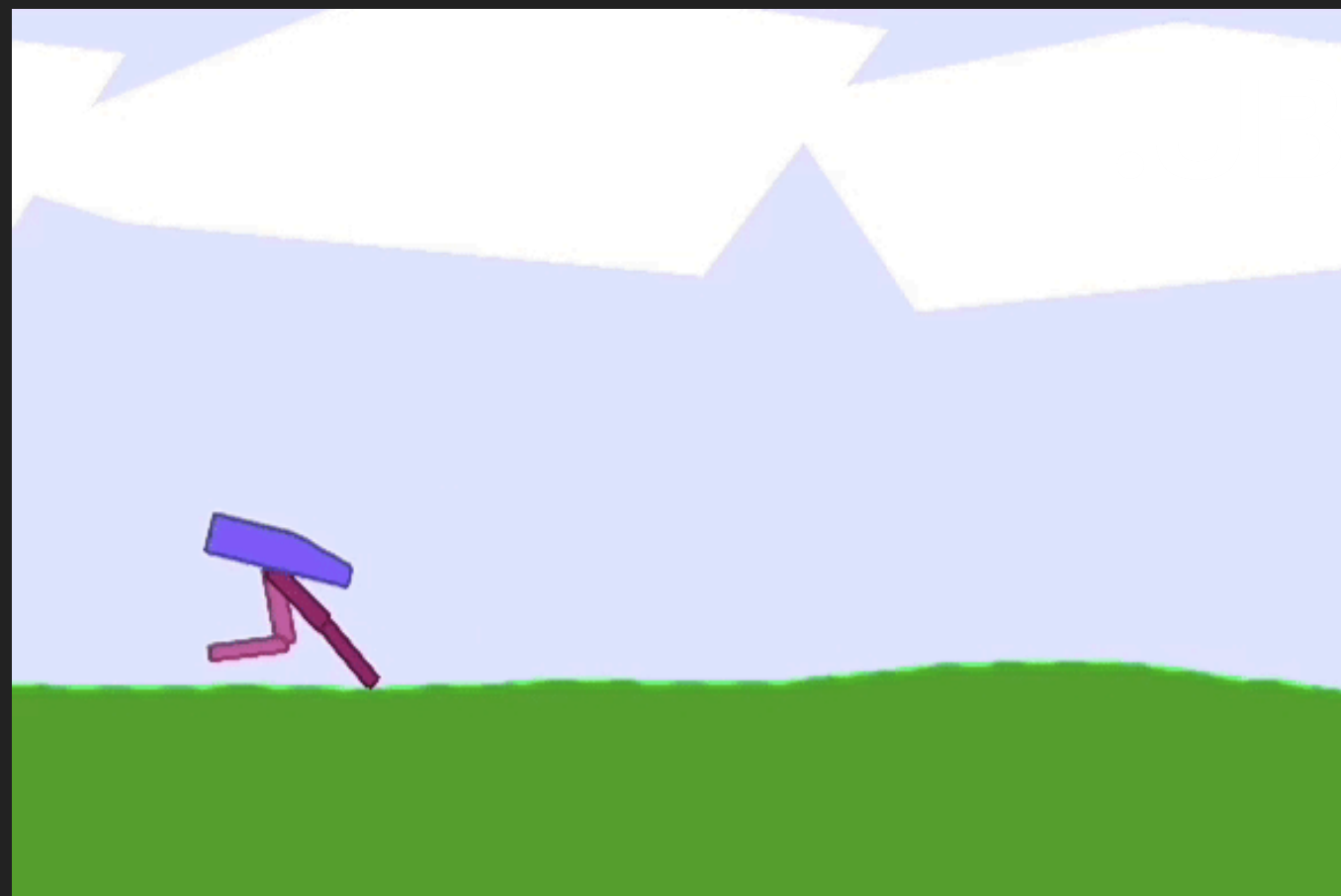
CUANDO USAR ACTOR-CRITIC?

ALGORITMO	ESTADO	ACCIÓN
Monte Carlo ES	Discreto	Discreto
Monte Carlo IS	Discreto	Discreto
SARSA	Discreto	Discreto
Q-Learning	Discreto	Discreto
DQN	Continuo	Discreto
Double DQN	Continuo	Discreto
Dueling DQN	Continuo	Discreto
REINFORCE	Continuo	Discreto / Continuo
Actor-Critic	Continuo	Discreto / Continuo



ACTOR-CRITIC (EJEMPLO)

- ▶ Actor-Critic one-step aplicado a BipedalWalker de Gymnasium.



CARACTERÍSTICAS

- ▶ Objetivo: aprender a caminar hacia adelante sin caerse, superando obstáculos en un terreno irregular.
- ▶ Espacio de estados (usa Lidar, Laser Distance Sensor, que mide la forma del terreno):
 - ✓ Posición y ángulo del torso.
 - ✓ Velocidades articulares (hombros y rodillas).
 - ✓ Contacto con el suelo (para detectar caídas).
- ▶ Espacio de acciones (tiene 4 articulaciones, 2 en cada pierna: cadera y rodilla):
 - ✓ Acciones son valores continuos en $[-1, 1]$, donde:
 - ✓ 1 = máxima fuerza en una dirección.
 - ✓ -1 = máxima fuerza en la dirección opuesta.



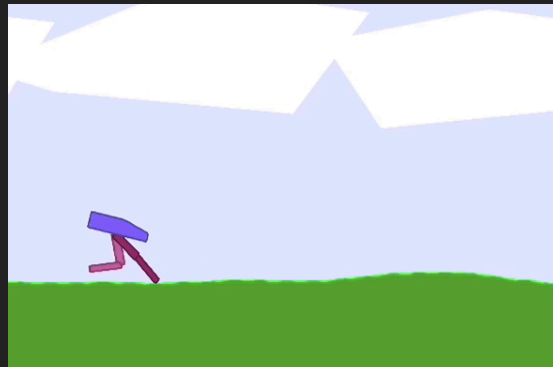
CARACTERÍSTICAS

► Recompensas y Penalizaciones

- ✓ Recompensa positiva: Avanzar hacia adelante sin caerse.
- ✓ Caer (ángulo del torso fuera de límites) -100.
- ✓ Consumo excesivo de energía (fuerza aplicada a los motores).
- ✓ Recompensa total = distancia recorrida - penalización por energía.

► El episodio termina si:

- ✓ El torso toca el suelo (ángulo $> 30^\circ$).
- ✓ Se alcanza el límite de pasos (1600).



PSEUDOCÓDIGO

Crear Actor

Crear Crítico

Bucle (por cada episodio)

 Inicializar estado

 Inicializar recompensa_acumulada = 0

 Bucle (por cada paso)

 Acción <- actor(estado) # Política: elegir acción

 Nuevo_estado, recompensa <- entorno(acción) # Tomar acción en el entorno

 Recompensa_acumulada <- Recompensa_acumulada + recompensa

 Valor <- critico(estado) # Estimar $V(s)$ del estado actual

 Valor_nuevo <- critico(nuevo_estado) # Estimar $V(s')$ del nuevo estado

 Ventaja <- recompensa + γ * Valor_nuevo - Valor # Calcular la ventaja (TD error)

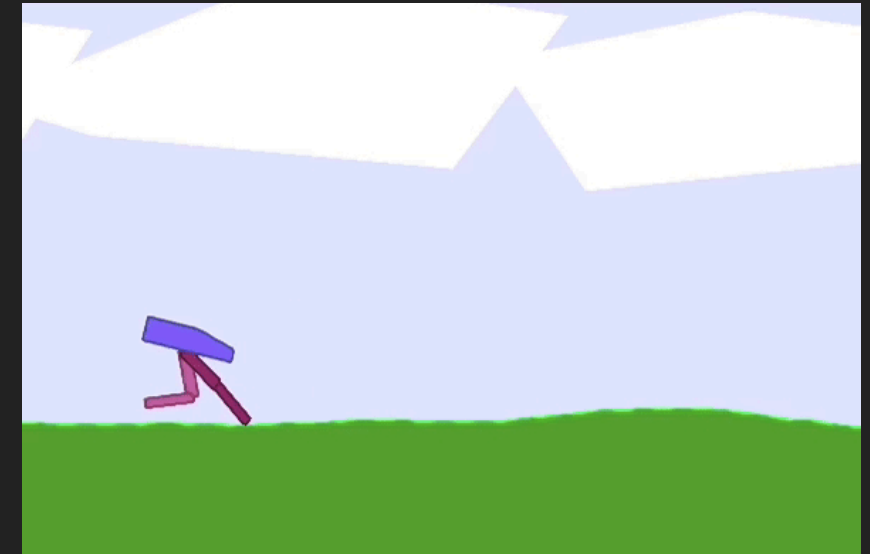
 critico.actualizar(estado, recompensa + γ * Valor_nuevo) # Actualizar el Crítico para minimizar el error TD

 actor.actualizar(estado, acción, Ventaja) # Actualizar el Actor usando la ventaja

 estado <- nuevo_estado # Pasar al siguiente estado

 Fin Bucle (pasos)

Fin Bucle (episodios)



.UBAfiuba

FACULTAD DE INGENIERÍA

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8, 229-256.
- ▶ R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- ▶ <https://www.kaggle.com/discussions/getting-started/162377>
- ▶ Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016, June). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995-2003). PMLR.
- ▶ Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., ... & Kavukcuoglu, K. (2016, June). Asynchronous methods for deep reinforcement learning. In *International conference on machine learning* (pp. 1928-1937). PmLR.
- ▶ Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018, July). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning* (pp. 1861-1870). Pmlr.
- ▶ Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., ... & Levine, S. (2019). Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*.
- ▶ https://github.com/ray-project/ray/tree/master/rllib/tuned_examples/sac